# CE385

## RESTful API, Validation และ ORM Prisma กับ PostgreSQL

RESTful API · Validation · Prisma ORM · PostgreSQL

**สัปดาห์ที่ 8 · คาบออนไลน์**

ภาคการศึกษาที่ 1 ปีการศึกษา 2569 · 3 หน่วยกิต

อาจารย์สนายุ จินตนาวรรณกุล

# ทบทวนสัปดาห์ที่แล้วสั้น ๆ

| หัวข้อ | ประเด็นที่ต้องจำให้ได้ |
|---|---|
| **Express** | `app.get/post/...` จับคู่ method + path เข้ากับ handler |
| **Middleware** | ฟังก์ชัน `(req, res, next)` ต่อกันเป็นสายพาน · ลืม `next()` คำขอค้าง |
| **4 ระดับ** | built-in · application · router · error-handling (4 พารามิเตอร์) |
| **รับข้อมูล** | `req.params` · `req.query` · `req.body` (ต้องมี `express.json()` ) |
| **แยกชั้น** | `server` → `app` → `routes` → `services` → `data` |
| **กับดัก** | `async` handler ที่ reject ต้องห่อด้วย `asyncHandler` ไม่งั้นหาย |

> **สัปดาห์ที่แล้วข้อมูลของเราอยู่ในอาร์เรย์ในหน่วยความจำ** — ปิดโปรแกรมทีเดียวหายหมด
> สัปดาห์นี้เราจะ **ยกข้อมูลชุดเดียวกันนั้นลงฐานข้อมูลจริง** และทำให้ API ป้องกันตัวเองได้

# วันนี้เราจะคุยอะไรกัน

**บรรยายก่อนครึ่งคาบ แล้วลงมือทีละขั้นอีกครึ่งคาบ** — จบคาบต้องมี API ที่ต่อฐานข้อมูลจริงและยิงผ่านทั้ง 5 เส้น

**ภาค ก — บรรยาย (ยังไม่ต้องเปิดโปรแกรม)**

1. **REST คืออะไร** — ข้อจำกัด 6 ข้อ และเหตุผลที่มันยังอยู่มา 20 ปี
2. **การจำลองทรัพยากร** — คิดเป็นคำนาม ไม่ใช่คำกริยา
3. **Method และ Status Code** — เลือกให้ถูกความหมาย · Idempotency
4. **Query Parameter และ Versioning** — filter · sort · paginate · `/api/v1`
5. **Validation** — Fail-fast และแนวคิด **ป้องกัน 3 ชั้น**
6. **ORM** — คืออะไร ข้อดี ข้อเสีย และเมื่อไรควรเขียน SQL เอง
7. **Prisma** — schema → migrate → generate → client

**ภาค ข — ลงมือ 11 ขั้น** ตั้งแต่ติดตั้ง PostgreSQL จนยิง API ผ่านทุกเส้น

# สัปดาห์นี้ตอบผลลัพธ์การเรียนรู้ข้อไหน

| CLO | คำอธิบาย | ส่วนที่เกี่ยวข้องวันนี้ |
|---|---|---|
| **CLO 1** | อธิบายหลักการและรูปแบบสถาปัตยกรรมแอปพลิเคชันได้ | ภาค ก หัวข้อ 1–2 · 6 |
| **CLO 3** | ออกแบบและพัฒนา RESTful API พร้อมจัดการข้อผิดพลาดได้ | ภาค ก หัวข้อ 3–5 · ภาค ข ทั้งหมด |
| **CLO 4** | ออกแบบสถาปัตยกรรมแบบชั้นและแยกความรับผิดชอบได้ | ภาค ข ขั้น 6–9 |
| **CLO 6** | เชื่อมต่อและจัดการฐานข้อมูลผ่าน ORM ได้ | ภาค ก หัวข้อ 7 · ภาค ข ขั้น 2–4 |

## ปรัชญาของวิชานี้ (เหมือนเดิมทุกสัปดาห์)

> **สถาปัตยกรรมนำ · โค้ดตาม**

วันนี้ "สถาปัตยกรรม" คือ **สัญญาที่ระบบเราให้ไว้กับโลกภายนอก** และ **ชั้นป้องกันที่ทำให้สัญญานั้นไม่ถูกละเมิด** เข้าใจสองเรื่องนี้แล้ว Zod กับ Prisma จะเหลือแค่ **ไวยากรณ์**

# คาบนี้เรียนออนไลน์ — อ่านตรงนี้ก่อน

## สไลด์ชุดนี้ออกแบบให้ "อ่านเองแล้วทำตามได้จนจบ"

| สิ่งที่ต้องรู้ | รายละเอียด |
|---|---|
| **โค้ดครบทุกไฟล์** | ทุกไฟล์ในโปรเจกต์มีโค้ด **เต็มไฟล์** อยู่ในสไลด์ ไม่มี `...` ให้เดาเอง |
| **ทำตามลำดับเท่านั้น** | ขั้นที่ 0 → 11 ห้ามข้าม เพราะแต่ละขั้นต่อยอดจากขั้นก่อน |
| **ฉบับ HTML กดคัดลอกโค้ดได้** | มีปุ่ม **คัดลอก** มุมขวาของทุกกล่องโค้ด · กด `?` ดูปุ่มลัดทั้งหมด |
| **ฉบับ PDF อ่านได้อย่างเดียว** | เฉลยทุกข้อพิมพ์เปิดไว้แล้ว เอาไว้อ่านทบทวน |
| **ถ้าติด** | ไปที่สไลด์ **ตารางแก้ปัญหา** ท้ายเล่มก่อน — 90% ของอาการที่เจอมีอยู่ในนั้น |

## ปุ่มลัดของสไลด์ฉบับ HTML

| ปุ่ม | ทำอะไร | ปุ่ม | ทำอะไร |
|---|---|---|---|
| `→` `Space` | สไลด์ถัดไป | `O` | ดูภาพรวมทุกสไลด์ |
| `←` | สไลด์ก่อนหน้า | `D` | สลับโหมดมืด/สว่าง |
| `Home` `End` | หน้าแรก / หน้าสุดท้าย | `F` | เต็มจอ |
| `?` | วิธีใช้ | `N` | โน้ตผู้สอน |

> **ตำแหน่งสไลด์ที่อ่านค้างไว้จะถูกจำไว้ในเบราว์เซอร์** — ปิดแล้วเปิดใหม่จะกลับมาที่เดิม

# ภาค ก · บรรยาย

**เข้าใจก่อน แล้วค่อยพิมพ์**

**ยังไม่ต้องเปิดโปรแกรม** — ช่วงนี้ขอให้ฟังและคิดตามอย่างเดียว

# 1 · REST คืออะไร

**ข้อจำกัด 6 ข้อ และเหตุผลที่มันยังอยู่มา 20 ปี**

# API คือสัญญา ไม่ใช่โค้ด

**API** = ข้อตกลงว่า "ถ้าคุณส่งอะไรมาแบบนี้ ผมจะตอบอะไรกลับไปแบบนั้น"

### สิ่งที่อยู่ในสัญญา

- ที่อยู่ (URL) ของแต่ละทรัพยากร
- คำกริยา (HTTP method) ที่ใช้ได้
- รูปร่างข้อมูลที่ส่งเข้า
- รูปร่างข้อมูลที่ตอบกลับ
- **รหัสสถานะ** ที่แปลว่าอะไร
- **รูปแบบข้อผิดพลาด**

### สิ่งที่ไม่อยู่ในสัญญา

- ใช้ภาษาอะไรเขียน
- ใช้ฐานข้อมูลตัวไหน
- โค้ดแบ่งกี่ไฟล์
- รันบนเครื่องอะไร

> **เมื่อสัญญาถูกประกาศออกไปแล้ว การแก้สัญญาแบบไม่เข้ากันกับของเดิม = ทำให้ระบบคนอื่นพัง**
> นี่คือเหตุผลที่เราต้องคิดเรื่อง **การออกแบบ** ก่อนลงมือเขียน ไม่ใช่เขียนไปแก้ไป

# ข้อจำกัด 6 ข้อของสถาปัตยกรรมแบบ REST

REST ไม่ใช่เทคโนโลยี แต่เป็น **ชุดข้อจำกัด** ที่ Roy Fielding เสนอไว้เมื่อปี 2000

| ข้อจำกัด | หมายความว่า | เราได้อะไรจากมัน |
|---|---|---|
| **Client–Server** | แยกฝั่งขอกับฝั่งให้บริการออกจากกัน | เปลี่ยนหน้าเว็บได้โดยไม่แตะ API |
| **Stateless** | เซิร์ฟเวอร์ไม่จำอะไรระหว่างคำขอ | เพิ่มเครื่องได้เรื่อย ๆ · คำขอไหนตกไปก็ยิงใหม่ได้ |
| **Cacheable** | ตอบได้ว่าคำตอบนี้เก็บไว้ใช้ซ้ำได้ไหม | ลดภาระเซิร์ฟเวอร์ลงมหาศาล |
| **Uniform Interface** | ทุกทรัพยากรใช้กติกาชุดเดียวกัน | รู้เส้นหนึ่ง = เดาเส้นที่เหลือได้ถูก |
| **Layered System** | แทรกชั้นกลางได้ (nginx · cache · gateway) | เพิ่มความปลอดภัย/ความเร็วโดยไม่แก้โค้ด |
| **Code on Demand** | ส่งโค้ดไปให้ client รันได้ *(ไม่บังคับ)* | แทบไม่มีใครใช้ในทางปฏิบัติ |

**ข้อที่เราจะได้สัมผัสจริงวันนี้คือ Stateless และ Uniform Interface**

อีก 4 ข้อจะเจอเต็ม ๆ ในสัปดาห์ที่ 10–12 (ความปลอดภัย · Redis · nginx)

# Stateless หมายถึงอะไรจริง ๆ

**ไม่ได้หมายความว่า "ไม่เก็บข้อมูล"** — ฐานข้อมูลก็ยังเก็บเหมือนเดิม

หมายความว่า **เซิร์ฟเวอร์ไม่จำว่าคุณเคยคุยอะไรไว้ในคำขอก่อนหน้า**

```
❌ แบบมีสถานะ (stateful)
   คำขอ 1: POST /login          → เซิร์ฟเวอร์จำไว้ในหน่วยความจำว่า "คนนี้คือ 6601"
   คำขอ 2: GET  /my-profile     → เซิร์ฟเวอร์เปิดดูความจำของตัวเอง

✅ แบบไร้สถานะ (stateless)
   คำขอ 1: POST /login          → ตอบ token กลับไปให้ client เก็บ
   คำขอ 2: GET  /students/6601  + Authorization: Bearer <token>
                                → ทุกอย่างที่ต้องรู้ อยู่ในคำขอใบนี้ใบเดียว
```

**ทำไมสำคัญ:** ถ้าเซิร์ฟเวอร์ไม่จำอะไร เราก็เอาเครื่องที่ 2, 3, 10 มาต่อขนานกันได้เลย คำขอจะไปลงเครื่องไหนก็ตอบได้เหมือนกันหมด — นี่คือรากของการขยายระบบทั้งหมด

# Uniform Interface — ข้อที่ให้ประโยชน์มากที่สุด

**ใช้กติกาชุดเดียวกันกับทุกทรัพยากร** แล้วคนที่มาใช้ API ของเราจะ **เดาถูก**

## ❌ ทุกทรัพยากรมีกติกาของตัวเอง

```
GET  /getAllStudents
POST /student/create
POST /updateStudentById?id=3
GET  /removeStudent/3
GET  /courses/list
POST /course-new
```

ต้องเปิดเอกสารทุกครั้งที่จะเรียกเส้นใหม่
เพราะ **เดาไม่ได้เลย**

## ✅ กติกาชุดเดียวทั้งระบบ

```
GET    /students
POST   /students
PATCH  /students/:id
DELETE /students/:id
GET    /courses
POST   /courses
```

เห็น `/students` แล้ว **เดา** `/courses` **ได้ถูกทันที**
เพราะกติกาเหมือนกัน

> **ความสม่ำเสมอมีค่ามากกว่าความถูกต้องสมบูรณ์แบบ**
> API ที่ตั้งชื่อ "ไม่สวยที่สุด" แต่สม่ำเสมอทั้งระบบ ใช้งานง่ายกว่า API ที่แต่ละเส้นสวยคนละแบบ

# 2 · การจำลองทรัพยากร

**คิดเป็นคำนาม ไม่ใช่คำกริยา**

# คำกริยาอยู่ที่ method · คำนามอยู่ที่ URL

HTTP มีคำกริยาให้ใช้อยู่แล้ว (`GET` `POST` `PATCH` `DELETE`)
เราจึง **ไม่ต้องใส่คำกริยาลงใน URL ซ้ำอีก**

| ❌ แบบที่ไม่ควรทำ | ✅ แบบที่ควรทำ | เพราะอะไร |
|---|---|---|
| `GET /getStudents` | `GET /students` | `GET` บอกว่า "เอามาดู" อยู่แล้ว |
| `POST /createStudent` | `POST /students` | `POST` บอกว่า "สร้างใหม่" อยู่แล้ว |
| `POST /deleteStudent/3` | `DELETE /students/3` | มี method สำหรับลบอยู่แล้ว ไม่ต้องใช้ POST |
| `GET /student` | `GET /students` | ทรัพยากรเป็นกลุ่ม ใช้พหูพจน์ให้เหมือนกันทั้งระบบ |
| `GET /students/3/getCourses` | `GET /students/3/courses` | คำนามซ้อนคำนาม อ่านเป็นประโยคได้ |
| `POST /studentUpdateScore` | `PATCH /students/3` | แก้บางฟิลด์ = `PATCH` |

> **วิธีตรวจง่าย ๆ:** อ่าน URL ออกเสียงแล้วต้องได้ **คำนามเรียงกัน** เท่านั้น
> ถ้ามีคำกริยาโผล่มา แปลว่าออกแบบผิด

# เมื่อไรจึงซ้อนทรัพยากร และซ้อนลึกแค่ไหน

**ซ้อนเมื่อของข้างในไม่มีความหมายถ้าไม่มีของข้างนอก**

```
GET /students/6601/enrollments      ← การลงทะเบียนของนักศึกษาคนนี้ (ซ้อนมีเหตุผล)
GET /courses/CE385/enrollments      ← การลงทะเบียนในวิชานี้ (ซ้อนมีเหตุผล)
```

**ห้ามซ้อนเกิน 2 ชั้น** — ลึกกว่านั้นให้ใช้ query parameter แทน

```
❌ GET /universities/1/faculties/2/students/6601/enrollments/9/payments
✅ GET /payments?enrollmentId=9
```

> **เหตุผล:** URL ที่ซ้อนลึกผูกโครงสร้างข้อมูลของเราไว้กับสัญญาที่ให้คนอื่น
> วันที่เราย้ายตารางหรือเปลี่ยนความสัมพันธ์ URL ทั้งชุดจะพังตามไปด้วย

# ทรัพยากรของเราวันนี้: `students`

ทั้งคาบเราจะทำ **CRUD ของทรัพยากรเดียว** ให้ครบและถูกต้องทุกรายละเอียด
ดีกว่าทำ 10 ทรัพยากรแบบครึ่ง ๆ กลาง ๆ

| ฟิลด์ | ชนิด | กติกา |
|---|---|---|
| `id` | UUID | ระบบสร้างให้ · ห้ามส่งมาจากภายนอก |
| `code` | ข้อความ 8–10 ตัวเลข | **ห้ามซ้ำ** — รหัสนักศึกษา |
| `name` | ข้อความ 2–100 ตัวอักษร | จำเป็น |
| `email` | อีเมล | **ห้ามซ้ำ** |
| `score` | จำนวนเต็ม 0–100 | ไม่ส่งมา = `0` |
| `active` | จริง/เท็จ | ไม่ส่งมา = `true` |
| `createdAt` · `updatedAt` | เวลา | ระบบใส่ให้เอง · ห้ามแก้จากภายนอก |

> **สังเกตคอลัมน์ขวา** — ตารางนี้จะกลายเป็น **โค้ด Zod** ในภาค ข แบบบรรทัดต่อบรรทัด
> การออกแบบที่ดีทำให้ขั้นเขียนโค้ดเหลือแค่การแปลภาษา

# 3 · Method และ Status Code

เลือกให้ถูกความหมาย

# HTTP Method ที่ต้องใช้ให้ถูก

| Method | ใช้เมื่อ | มี body ไหม | ยิงซ้ำแล้วผลเหมือนเดิม | ปลอดภัย |
|---|---|---|---|---|
| **GET** | อ่านข้อมูล | ไม่มี | ✅ ใช่ | ✅ ไม่เปลี่ยนอะไร |
| **POST** | สร้างของใหม่ | มี | ❌ **ไม่** — ยิง 3 ครั้งได้ 3 แถว | ❌ |
| **PUT** | แทนที่ทั้งก้อน | มี | ✅ ใช่ | ❌ |
| **PATCH** | แก้บางฟิลด์ | มี | ⚠️ ขึ้นกับการออกแบบ | ❌ |
| **DELETE** | ลบ | ไม่มี | ✅ ใช่ *(ครั้งที่สองได้ 404)* | ❌ |

## `PUT` กับ `PATCH` ต่างกันตรงไหน

```
PUT   /students/3   { "code":"66010001", "name":"สมชาย", "email":"...", "score":80, "active":true }
                    ↑ ต้องส่งมาให้ครบทุกฟิลด์ — ฟิลด์ที่ไม่ส่งจะถูกล้างเป็นค่าว่าง

PATCH /students/3   { "score": 80 }
                    ↑ ส่งมาแค่ที่จะแก้ — ฟิลด์อื่นคงเดิม
```

**วันนี้เราเลือก** `PATCH` เพราะในงานจริง 90% ของการแก้ข้อมูลคือการแก้ไม่กี่ฟิลด์

# Idempotency — ยิงซ้ำแล้วเกิดอะไรขึ้น

**Idempotent** = ยิงคำขอเดิมซ้ำกี่ครั้ง **สถานะปลายทางเหมือนกันหมด**

```
DELETE /students/3   ครั้งที่ 1 → 204 (ลบสำเร็จ)
DELETE /students/3   ครั้งที่ 2 → 404 (ไม่มีแล้ว)
      ↑ status ต่างกัน แต่ "สถานะของฐานข้อมูล" เหมือนกัน = ยังนับว่า idempotent

POST /students  { "code":"66010001", ... }  ครั้งที่ 1 → 201 สร้างแถวที่ 1
POST /students  { "code":"66010001", ... }  ครั้งที่ 2 → 409 เพราะ code ซ้ำ
      ↑ ถ้าไม่มี @unique กั้นไว้ จะได้ 2 แถวซ้ำกัน = ไม่ idempotent
```

**ทำไมเรื่องนี้สำคัญกับงานจริง:** เน็ตหลุดตอนกำลังส่งคำขอ แล้วมือถือยิงซ้ำอัตโนมัติ
ถ้าเส้นนั้นไม่ idempotent ผู้ใช้จะได้รายการซ้ำ หรือถูกตัดเงินสองครั้ง
**ข้อจำกัด `@unique` ในฐานข้อมูลที่เราจะใส่วันนี้ คือสิ่งที่กันเรื่องนี้จริง ๆ**

# Status Code — กลุ่ม 2xx สำเร็จ

| รหัส | ชื่อ | ใช้เมื่อ | จุดที่คนพลาด |
|---|---|---|---|
| **200** | OK | อ่านสำเร็จ · แก้สำเร็จ | ใช้ถูกกันเกือบทุกคน |
| **201** | Created | **สร้างของใหม่สำเร็จ** | คนตอบ 200 แทน 201 บ่อยมาก |
| **204** | No Content | ลบสำเร็จ · ไม่มีอะไรจะส่งกลับ | **ห้ามใส่ body ใน 204** (ผิดมาตรฐาน) |

## `201` ต้องมาพร้อม header `Location`

```
POST /api/v1/students          → 201 Created
                                  Location: /api/v1/students/3f0c1e8a-...

                                  { "data": { "id": "3f0c1e8a-...", ... } }
```

`Location` บอก client ว่า **ของใหม่ที่เพิ่งสร้างอยู่ที่ไหน** — ธรรมเนียมที่คนลืมมากที่สุด

## ในโค้ดที่เราจะเขียนวันนี้

```
res.status(201).location(`/api/v1/students/${created.id}`).json({ data: created });
```

# Status Code — กลุ่ม 4xx ผู้ใช้ผิด · 5xx เราผิด

| รหัส | ชื่อ | ใช้เมื่อ | ตัวอย่างในโปรเจกต์วันนี้ |
|---|---|---|---|
| **400** | Bad Request | **คำขอเขียนมาผิดรูป** | `GET /students/abc` — `abc` ไม่ใช่ UUID |
| **401** | Unauthorized | ยังไม่ได้ยืนยันตัวตน | *(สัปดาห์ที่ 10)* |
| **403** | Forbidden | ยืนยันตัวตนแล้ว แต่ไม่มีสิทธิ์ | *(สัปดาห์ที่ 10)* |
| **404** | Not Found | ไม่มีทรัพยากรนี้ | `GET /students/<uuid ที่ไม่มีในตาราง>` |
| **409** | Conflict | **ชนกับสิ่งที่มีอยู่แล้ว** | `POST` ด้วย `code` หรือ `email` ที่ซ้ำ |
| **422** | Unprocessable Entity | **รูปแบบคำขอถูก แต่เนื้อหาใช้ไม่ได้** | `{"score": 150}` — เกิน 100 |
| **500** | Internal Server Error | **บั๊กของเรา** | `undefined is not a function` |
| **503** | Service Unavailable | บริการที่เราพึ่งพาล่ม | ต่อฐานข้อมูลไม่ได้ ( `P1001` ) |

**กฎที่ต้องจำ:** `4xx` = "คุณส่งมาผิด แก้แล้วยิงใหม่ได้" · `5xx` = "เราผิด คุณแก้อะไรก็ไม่ช่วย"

**ห้ามตอบ 200 พร้อมข้อความว่า error** — ทำแบบนั้นคือทำลาย Uniform Interface ทั้งหมด

# 400 กับ 422 ต่างกันตรงไหน

เส้นแบ่งที่ **โปรเจกต์นี้ใช้** และจะเขียนลงโค้ดจริง

## 400 Bad Request

**ผิดที่ path หรือ query**

```
GET /students/abc
        ↑ ตำแหน่งนี้ต้องเป็น UUID

GET /students?page=abc
                  ↑ ต้องเป็นตัวเลข

GET /students?limit=999
                  ↑ เกินเพดาน 100
```

คำขอ **เขียนมาผิดรูปตั้งแต่ URL**

## 422 Unprocessable Entity

**ผิดที่ body**

```
POST /students  {}
      ↑ URL ถูก แต่ไม่ส่งข้อมูลมา

POST /students  { "score": 150 }
      ↑ ชนิดถูก แต่ค่าเกินช่วง

POST /students  { "nickname": "x" }
      ↑ ฟิลด์ที่ระบบไม่รู้จัก
```

รูปแบบคำขอถูก แต่ **เนื้อหาใช้งานไม่ได้**

> ***สำคัญกว่าการเลือกให้ "ถูกตามตำรา" คือการเลือกแล้ว *ใช้เหมือนกันทั้งระบบ***
> เอกสาร API ต้องเขียนกำกับไว้ว่าโปรเจกต์นี้ตีความ 400/422 อย่างไร

# คิดก่อนดูเฉลย

ลองเลือก status code ที่ถูกต้องสำหรับแต่ละกรณี

| # | สถานการณ์ | ควรตอบอะไร |
|---|---|---|
| 1 | `POST /students` สร้างสำเร็จ | _____ |
| 2 | `POST /students` ด้วยอีเมลที่มีคนใช้แล้ว | _____ |
| 3 | `GET /students/<uuid ถูกรูปแบบ แต่ไม่มีในตาราง>` | _____ |
| 4 | `GET /students/hello` | _____ |
| 5 | `DELETE /students/<uuid จริง>` สำเร็จ | _____ |
| 6 | `POST /students` ด้วย `{"score": 150}` | _____ |
| 7 | ฐานข้อมูลถูกปิดอยู่ ต่อไม่ได้ | _____ |
| 8 | โค้ดเราเรียกฟังก์ชันที่ไม่มีอยู่ | _____ |

**หยุดคิดให้ครบ 8 ข้อก่อน** แล้วค่อยเลื่อนไปดูเฉลย

ข้อ 2 กับ 6 คือข้อที่คนตอบผิดมากที่สุด

# เฉลย

| # | สถานการณ์ | คำตอบ | เหตุผล |
|---|---|---|---|
| 1 | สร้างสำเร็จ | **201** | ต้องมี header `Location` ด้วย |
| 2 | อีเมลซ้ำ | **409** | ชนกับของที่มีอยู่ — ไม่ใช่ 422 เพราะข้อมูลที่ส่งมา *ถูกต้องทุกอย่าง* |
| 3 | uuid ถูก แต่ไม่มีแถว | **404** | ที่อยู่ถูก แต่ไม่มีของอยู่ที่นั้น |
| 4 | `hello` ไม่ใช่ uuid | **400** | คำขอเขียนมาผิดรูป — ไม่ใช่ 404 |
| 5 | ลบสำเร็จ | **204** | ห้ามใส่ body |
| 6 | `score` = 150 | **422** | รูปแบบถูก แต่ค่าใช้ไม่ได้ |
| 7 | ต่อฐานข้อมูลไม่ได้ | **503** | ไม่ใช่ความผิดผู้ใช้ และไม่ใช่บั๊กของโค้ด |
| 8 | บั๊กของเรา | **500** | และ **ห้ามส่งรายละเอียดบั๊กออกไปให้ client** |

> **ข้อ 2 vs ข้อ 6** — ทั้งคู่คือ "สร้างไม่สำเร็จ" แต่ต่างกันที่สาเหตุ
> 422 = ข้อมูลผิด แก้ข้อมูลแล้วยิงใหม่ได้ · 409 = ข้อมูลถูกแต่ที่นั่งถูกจองแล้ว ต้องเปลี่ยนค่า

# 4 · Query Parameter และ Versioning

**filter · sort · paginate · `/api/v1`**

# Query Parameter ทำหน้าที่ 3 อย่าง

```
GET /api/v1/students?q=สมชาย&sort=score&order=desc&page=2&limit=10
                    └ กรอง ─┘└──── เรียง ────┘└─ แบ่งหน้า ─┘
```

| กลุ่ม | พารามิเตอร์ | หน้าที่ |
|---|---|---|
| **กรอง (filter)** | `q` `active` `minScore` | ตัดข้อมูลที่ไม่ต้องการออก |
| **เรียง (sort)** | `sort` `order` | บอกว่าเรียงด้วยฟิลด์ไหน ขึ้นหรือลง |
| **แบ่งหน้า (paginate)** | `page` `limit` | เอามาทีละชุด ไม่เอาทั้งตาราง |

## กฎ 3 ข้อที่ต้องทำทุกครั้ง

1. **มีค่าเริ่มต้นให้ทุกตัว** — ไม่ส่งมาก็ต้องทำงานได้ (`page=1` `limit=10`)
2. **จำกัดเพดาน `limit`** — ไม่มีเพดาน = ใครก็ยิง `?limit=99999999` ถล่มฐานข้อมูลเราได้
3. **จำกัดค่าที่ `sort` รับได้** — ปล่อยอิสระ = เปิดช่องให้คนเรียงด้วยคอลัมน์ที่ไม่มี index

> ค่าที่มาจาก URL **เป็นข้อความล้วนทั้งหมด** — `?page=2` คือสตริง `"2"` ไม่ใช่เลข `2`
> จุดนี้คือเหตุผลที่โค้ดของเราต้องใช้ `z.coerce` ในภาค ข

# แบ่งหน้าแบบ offset เทียบกับ cursor

## offset/limit — แบบที่เราใช้วันนี้

```
GET /students?page=3&limit=10
   → ข้าม 20 แถว เอา 10 แถว
```

**ข้อดี** เข้าใจง่าย · กระโดดไปหน้าไหนก็ได้ · ทำหน้า 1 2 3 … ได้ตรงไปตรงมา

**ข้อเสีย** ยิ่งหน้าลึกยิ่งช้า (ฐานข้อมูลต้องนับข้ามทีละแถว) · ถ้ามีคนเพิ่มข้อมูลระหว่างที่เราเปิดหน้า 2 จะเห็นแถวซ้ำหรือแถวหาย

## cursor-based

```
GET /students?after=<id ล่าสุด>&limit=10
   → เอา 10 แถวถัดจาก id นี้
```

**ข้อดี** เร็วเท่ากันทุกหน้า · ไม่มีปัญหาแถวซ้ำ/หาย · เหมาะกับฟีดที่เลื่อนลงเรื่อย ๆ

**ข้อเสีย** กระโดดไปหน้า 50 ตรง ๆ ไม่ได้ · ทำ UI แบบเลขหน้าไม่ได้

> **เลือกอย่างไร:** ตารางผู้ใช้ในหลังบ้าน → **offset** · ฟีดที่เลื่อนไม่สิ้นสุด → **cursor**
> วันนี้ใช้ offset เพราะข้อมูลนักศึกษาเป็นตารางที่ต้องกระโดดหน้าได้

# ใส่เวอร์ชันไว้ใน path ตั้งแต่วันแรก

```
/api/v1/students      ← ทำแบบนี้ตั้งแต่เส้นแรกของโปรเจกต์
/students             ← วันที่ต้องแก้สัญญา จะไม่มีทางถอยได้เลย
```

**เหตุผล:** วันหนึ่งเราจะต้องแก้สัญญาแบบที่เข้ากับของเดิมไม่ได้ เช่น
เปลี่ยน `name` เป็น `firstName` + `lastName` หรือเปลี่ยนรูปแบบ error

ถ้ามี `/api/v1` อยู่แล้ว เราเปิด `/api/v2` ขึ้นมาขนานกันได้ **โดยที่ระบบเก่ายังใช้ต่อได้**

| การเปลี่ยนแปลง | เข้ากับของเดิมได้ไหม | ต้องขึ้นเวอร์ชันใหม่ไหม |
|---|---|---|
| เพิ่มฟิลด์ใหม่ในคำตอบ | ✅ ได้ | ไม่ต้อง |
| เพิ่ม query parameter ที่ไม่บังคับ | ✅ ได้ | ไม่ต้อง |
| **ลบฟิลด์ออกจากคำตอบ** | ❌ ไม่ได้ | **ต้อง** |
| **เปลี่ยนชื่อฟิลด์** | ❌ ไม่ได้ | **ต้อง** |
| **ทำให้ฟิลด์เดิมกลายเป็นฟิลด์บังคับ** | ❌ ไม่ได้ | **ต้อง** |

# 5 · Validation

**Fail-fast และแนวคิดป้องกัน 3 ชั้น**

# Fail-fast — ตีกลับให้เร็วที่สุด

**ยิ่งข้อมูลผิดเดินลึกเข้าไปในระบบ ราคาของการแก้ยิ่งแพงขึ้นเรื่อย ๆ**

| ตีกลับที่ | ราคา |
|---|---|
| **ที่ประตูหน้า** (validate) | ตอบ 422 · จบ · ไม่แตะฐานข้อมูลเลย |
| ที่ชั้นตรรกะ | เสียเวลาคำนวณไปแล้วบางส่วน |
| ที่ฐานข้อมูล | เปลือง connection · อาจล็อกแถวไว้แล้ว |
| **ไม่ตีกลับเลย** | **ข้อมูลเสียถูกบันทึกลงตาราง** — ต้องมานั่งไล่ล้างทีหลัง |

## กฎที่ใช้ได้ทั้งชีวิตการทำงาน

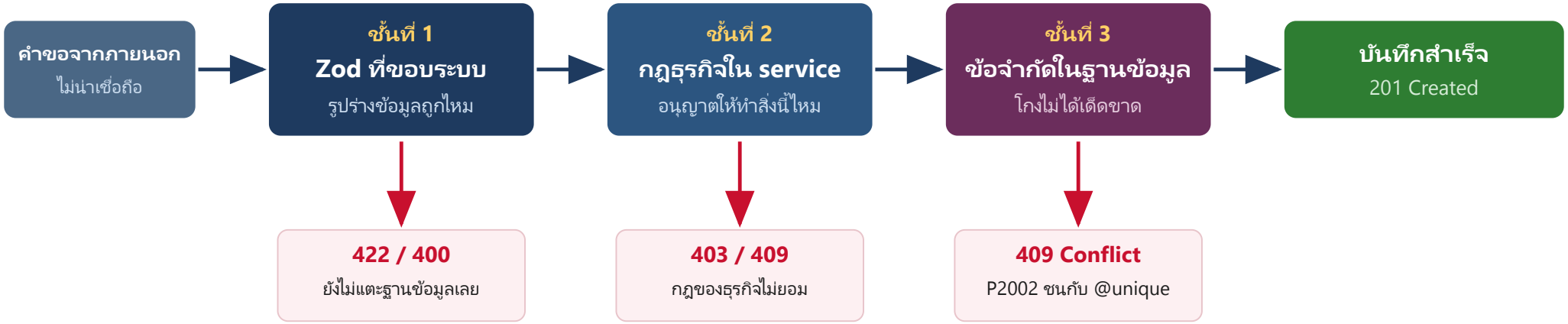
**อย่าเชื่อข้อมูลจากภายนอกแม้แต่ตัวอักษรเดียว**

ไม่ว่าจะมาจากหน้าเว็บที่เราเขียนเองก็เถอะ — เพราะใครก็ยิงตรงเข้า API ได้
โดยไม่ผ่านหน้าเว็บของเรา

# ป้องกัน 3 ชั้น — แกนเรื่องของคาบนี้

**ข้อมูลต้องผ่านทั้ง 3 ด่านจึงจะถูกบันทึก — ด่านไหนไม่ผ่านก็ตีกลับทันที**

ชั้นที่ 1 และ 2 คือโค้ดที่เราเขียน (พลาดได้) · ชั้นที่ 3 คือฐานข้อมูลบังคับ (พลาดไม่ได้)



# ควรตรวจอะไรบ้าง ที่ชั้นไหน

| ชั้น | เครื่องมือ | ตรวจอะไร | ตัวอย่าง |
|---|---|---|---|
| **1. ขอบระบบ** | Zod | **รูปร่างและช่วงค่า** | ส่ง `name` มาไหม · `score` อยู่ใน 0–100 ไหม · `email` รูปแบบถูกไหม |
| **2. ตรรกะธุรกิจ** | โค้ดใน `services/` | **กฎที่ต้องดูข้อมูลอื่นประกอบ** | นักศึกษาลงวิชานี้เกินจำนวนที่นั่งไหม · ลบได้ไหมถ้ายังมีหนี้ค้าง |
| **3. ฐานข้อมูล** | `@unique` `@db.VarChar` `NOT NULL` | **ความคงสภาพของข้อมูล** | `code` ห้ามซ้ำ · `name` ยาวไม่เกิน 100 |

## สองคำถามที่นักศึกษาถามเสมอ

**"ตรวจซ้ำ 3 ชั้นไม่เปลืองเหรอ"**

ไม่ — เพราะแต่ละชั้นตรวจ **เรื่องที่ชั้นอื่นตรวจแทนไม่ได้** และชั้นที่ 3 ยังกันกรณีที่ชั้น 1–2 มีบั๊กด้วย

**"ตรวจที่หน้าเว็บแล้ว ยังต้องตรวจที่ API อีกไหม"**

ต้อง — การตรวจที่หน้าเว็บมีไว้ให้ **ผู้ใช้สะดวก** ไม่ใช่เพื่อความปลอดภัย

ใครก็เปิด Postman ยิงตรงเข้า API ได้โดยไม่ผ่านหน้าเว็บของเรา

# รูปแบบข้อผิดพลาดต้องเหมือนกันทั้งระบบ

## ❌ แบบที่พบบ่อยในโปรเจกต์นักศึกษา — แต่ละเส้นตอบคนละแบบ

```
{ "message": "not found" }
{ "error": "invalid email" }
{ "success": false, "msg": "duplicate" }
{ "errors": [ "name required", "score too high" ] }
```

ฝั่งที่เรียกใช้ต้องเขียน `if` แยกทุกเส้น — และพลาดแน่นอนเมื่อมีเส้นใหม่

## ✅ รูปแบบชุดเดียวที่เราจะใช้วันนี้

```
{
  "error": {
    "code": "VALIDATION_ERROR",
    "message": "ข้อมูลที่ส่งมาไม่ถูกต้อง",
    "details": [
      { "field": "code",  "message": "ต้องระบุรหัสนักศึกษา" },
      { "field": "score", "message": "คะแนนต้องไม่เกิน 100" }
    ]
  }
}
```

`code` ให้โปรแกรมเทียบ · `message` ให้มนุษย์อ่าน · `details` ให้หน้าเว็บขึ้นข้อความใต้ช่องกรอกที่ผิด

**ทุกเส้น ทุกข้อผิดพลาด ใช้รูปนี้เหมือนกันหมด** — จึงต้องมีตัวดักรวมศูนย์ที่เดียว

# คิดก่อนดูเฉลย

โจทย์: ระบบลงทะเบียนเรียน มีกฎว่า **"หนึ่งวิชารับได้ไม่เกิน 40 คน"**

### คำถามที่ 1

กฎนี้ควรตรวจที่ชั้นไหน — Zod · service · หรือฐานข้อมูล?

### คำถามที่ 2

แล้วกฎ **"อีเมลต้องไม่ซ้ำกับคนอื่น"** ควรตรวจที่ชั้นไหน?

### คำถามที่ 3

ถ้าเราตรวจ "อีเมลไม่ซ้ำ" ด้วยการ `findUnique` ใน service ก่อนแล้วค่อย `create`
ยังจำเป็นต้องมี `@unique` ในฐานข้อมูลอีกไหม?

> **ข้อ 3 คือหัวใจของคาบนี้** — คิดให้ดีก่อนดูเฉลย
> คำใบ้: ลองนึกภาพว่ามีคนสองคนกดปุ่มสมัครพร้อมกันเสี้ยววินาทีเดียวกัน

# เฉลย

### ข้อ 1 — "หนึ่งวิชารับไม่เกิน 40 คน" → ชั้นที่ 2 (service)

เพราะต้อง **นับข้อมูลอื่นประกอบ** ก่อนตัดสิน Zod มองเห็นแค่คำขอใบนี้ใบเดียว จึงตรวจไม่ได้

ตอบ `409 Conflict` หรือ `422` ตามที่ตกลงกันไว้ในเอกสาร API

### ข้อ 2 — "อีเมลไม่ซ้ำ" → ชั้นที่ 3 (`@unique` ในฐานข้อมูล)

Zod ตรวจได้แค่ว่า *รูปแบบ* อีเมลถูก แต่ตรวจไม่ได้ว่าซ้ำกับใคร เพราะไม่รู้จักฐานข้อมูล

### ข้อ 3 — ยังจำเป็นอย่างยิ่ง ✅

```
เวลา 10:00:00.000  คำขอ A: findUnique("a@x.com") → null  "ว่าง เอาเลย"
เวลา 10:00:00.001  คำขอ B: findUnique("a@x.com") → null  "ว่าง เอาเลย"
เวลา 10:00:00.002  คำขอ A: create → สำเร็จ
เวลา 10:00:00.003  คำขอ B: create → ถ้าไม่มี @unique ก็ได้แถวซ้ำ
```

> **ช่องว่างระหว่าง "ตรวจ" กับ "เขียน" คือช่องที่ข้อมูลซ้ำแทรกเข้ามาได้**
>
> มีแต่ฐานข้อมูลที่ตัดสินได้ในจังหวะเดียวกับที่เขียน — **นี่คือเหตุผลที่ต้องมีชั้นที่ 3 เสมอ**

# 6 · ORM

**คืออะไร ข้อดี ข้อเสีย และเมื่อไรควรเขียน SQL เอง**

# ถ้าคุยกับฐานข้อมูลด้วย SQL ตรง ๆ

```
import { Client } from "pg";
const client = new Client({ connectionString: process.env.DATABASE_URL });

const { rows } = await client.query(
  "SELECT id, code, name, email, score FROM students WHERE name LIKE $1 ORDER BY score DESC LIMIT $2 OFFSET $3",
  [`%${q}%`, limit, (page - 1) * limit]
);
```

| ปัญหา | รายละเอียด |
|---|---|
| **TypeScript ไม่รู้อะไรเลย** | `rows` เป็น `any[]` — พิมพ์ `row.nmae` ผิดก็ไม่มีใครเตือน |
| **เขียนชื่อคอลัมน์เป็นข้อความ** | เปลี่ยนชื่อคอลัมน์ในตาราง ต้องไปไล่แก้ทุกสตริงเอง |
| **ผูก `$1 $2 $3` ด้วยลำดับ** | สลับลำดับพารามิเตอร์ = บั๊กที่หาสาเหตุยากที่สุดแบบหนึ่ง |
| **ต่อสตริง SQL เอง** | เผลอต่อค่าจากผู้ใช้เข้าไปตรง ๆ = **SQL Injection** |
| **ไม่มีประวัติการแก้โครงสร้าง** | ตารางบนเครื่องเรากับบนเซิร์ฟเวอร์ไม่เหมือนกันโดยไม่มีใครรู้ |

> ทุกข้อข้างบน **แก้ด้วยความขยันไม่ได้** — ต้องแก้ด้วยเครื่องมือ

# ORM คืออะไร

**ORM (Object–Relational Mapping)** = ตัวกลางที่ทำให้เราคุยกับฐานข้อมูล
ด้วย **ออบเจกต์ในภาษาที่เราเขียน** แทนการเขียน SQL เป็นสตริง

```
// เราเขียนแบบนี้                          ORM แปลเป็น SQL ให้เอง
await prisma.student.findMany({               // SELECT * FROM students
  where: { score: { gte: 80 } },              // WHERE score >= 80
  orderBy: { score: "desc" },                 // ORDER BY score DESC
  take: 10,                                   // LIMIT 10
});
```

| ✅ ข้อดี | ❌ ข้อเสีย |
|---|---|
| TypeScript รู้ชื่อฟิลด์และชนิดข้อมูลครบ | มีชั้นเพิ่มขึ้นมาให้ต้องเรียนรู้ |
| พิมพ์ชื่อฟิลด์ผิด = แดงทันทีตอนเขียน | query ที่ซับซ้อนมาก ๆ เขียนด้วย ORM ยากกว่า SQL |
| กัน SQL Injection ให้โดยปริยาย | ถ้าไม่ระวังจะเผลอยิง query เกินจำเป็น (**ปัญหา N+1**) |
| มีระบบ migration ติดมาด้วย | บางครั้งมันแปลออกมาเป็น SQL ที่ไม่ใช่ที่เราอยากได้ |
| ย้ายฐานข้อมูลได้ง่ายขึ้น | ต้องพึ่งพาเครื่องมือของคนอื่น |

# แล้วเมื่อไรควรเขียน SQL เอง

**ใช้ ORM เป็นค่าตั้งต้น** แล้วลงไปเขียน SQL เองเฉพาะจุดที่จำเป็น

| สถานการณ์ | ทางเลือก |
|---|---|
| CRUD ทั่วไป · ค้นหา · แบ่งหน้า | **ORM** — 95% ของงานอยู่ในกลุ่มนี้ |
| รายงานที่มี `GROUP BY` ซ้อน `JOIN` หลายชั้น | **SQL เอง** ผ่าน `prisma.$queryRaw` |
| ใช้ความสามารถเฉพาะของ PostgreSQL (full-text search · window function) | **SQL เอง** |
| ต้องปรับจูนประสิทธิภาพระดับ query plan | **SQL เอง** แล้วดูด้วย `EXPLAIN ANALYZE` |
| ย้ายข้อมูลครั้งใหญ่ทีละล้านแถว | **SQL เอง** หรือเครื่องมือเฉพาะทาง |

```
// Prisma เปิดประตูหลังไว้ให้ ใช้เมื่อจำเป็นจริง ๆ
const report = await prisma.$queryRaw`
  SELECT active, COUNT(*) AS total, AVG(score) AS avg_score
  FROM students GROUP BY active`;
```

> **ORM ไม่ใช่สิ่งที่มาแทน SQL** — คนที่ใช้ ORM ได้ดีคือคนที่อ่าน SQL ที่มันสร้างออกมาได้

# 7 · Prisma

**schema → migrate → generate → client**

# Prisma ประกอบด้วย 3 ชิ้น

| ชิ้น | คืออะไร | ไฟล์/คำสั่ง |
|---|---|---|
| **Prisma Schema** | ไฟล์เดียวที่บอกว่าฐานข้อมูลมีตารางอะไร ฟิลด์อะไร | `prisma/schema.prisma` |
| **Prisma Migrate** | เครื่องมือแปลง schema เป็นคำสั่ง SQL และเก็บเป็นประวัติ | `npx prisma migrate dev` |
| **Prisma Client** | โค้ด TypeScript ที่ถูกสร้างขึ้น **ตามตารางของเราเอง** | `npx prisma generate` |

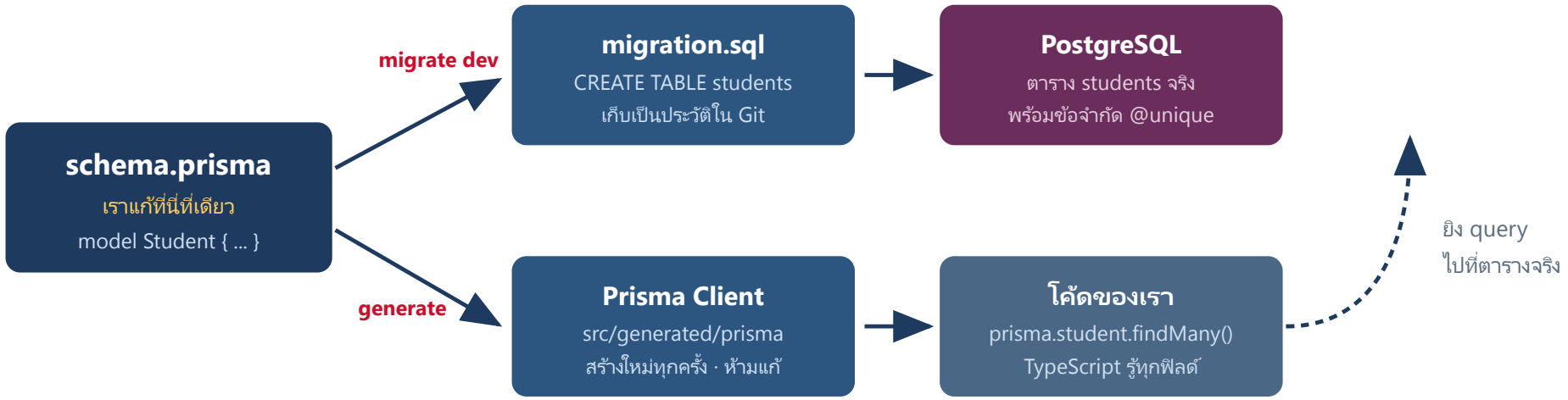
## จุดที่ทำให้ Prisma ต่างจาก ORM ตัวอื่น

> **Prisma Client ไม่ได้เขียนไว้ล่วงหน้า — มันถูกสร้างใหม่ทุกครั้งจาก schema ของเรา**

ผลคือ `prisma.student` มีอยู่จริงในระบบชนิดข้อมูลของ TypeScript
พิมพ์ `prisma.studnet` หรือ `where: { scoer: 80 }` จะ **แดงทันทีตอนเขียน** ไม่ต้องรอรัน

และเมื่อเราเพิ่มฟิลด์ใน schema แล้วสั่ง generate ใหม่
**ทุกที่ในโปรเจกต์ที่ต้องแก้ตามจะแดงให้เห็นหมด** — คอมไพเลอร์กลายเป็นรายการสิ่งที่ต้องทำ

# วงจรการทำงานของ Prisma




**แก้ schema แล้วต้องทำทั้งสองทาง — migrate ให้ฐานข้อมูลตรง และ generate ให้ TypeScript ตรง**

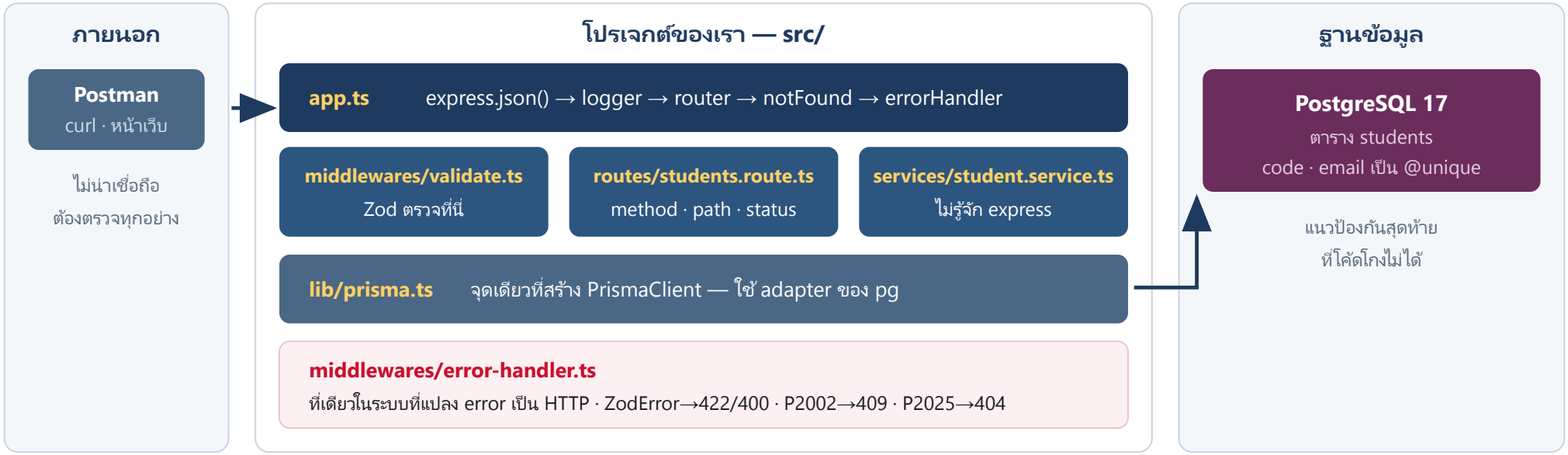
# Migration คือ Git ของฐานข้อมูล

```
prisma/migrations/
├─ 20260927070235_init/migration.sql              ← สร้างตาราง students
├─ 20261004093012_add_phone_to_student/migration.sql   ← เพิ่มคอลัมน์ phone
└─ 20261011140533_make_email_optional/migration.sql    ← แก้ข้อจำกัด
```

| ถ้าไม่มี migration | ถ้ามี migration |
|---|---|
| เพื่อนในกลุ่มต้องมาถามว่า "เพิ่มคอลัมน์อะไรบ้าง" | `git pull` แล้ว `prisma migrate dev` — จบ |
| ตารางบนเครื่องเรากับบนเซิร์ฟเวอร์ไม่เหมือนกัน | ทุกที่มีลำดับการเปลี่ยนแปลงชุดเดียวกัน |
| ย้อนกลับไม่ได้ ไม่รู้ว่าเคยเปลี่ยนอะไรไป | เปิดอ่านประวัติได้ทุกบรรทัด |
| โค้ดใหม่รันบนตารางเก่า = พังตอนขึ้นระบบจริง | ขึ้นระบบพร้อมกันได้ทั้งโค้ดและตาราง |

**กฎเหล็ก:** โฟลเดอร์ `prisma/migrations/` **ต้อง commit ขึ้น Git ทุกครั้ง**
และ **ห้ามแก้ไฟล์ migration ที่ถูก apply ไปแล้ว** — ถ้าผิดให้สร้าง migration ใหม่มาแก้

# สถาปัตยกรรมที่เราจะสร้างในภาค ข




**แต่ละกล่องคือหนึ่งไฟล์ · แต่ละไฟล์มีหน้าที่เดียว · ภาค ข จะเขียนไล่จากล่างขึ้นบน**

# ภาค ข · ลงมือ

**11 ขั้น จากเครื่องเปล่าถึง API ที่ยิงผ่าน**

**เปิดโปรแกรมแล้วพิมพ์ตามไปด้วย** — ห้ามข้ามขั้น

# แผนที่ของภาค ข

| ขั้น | ทำอะไร | ไฟล์ที่เกิดขึ้น |
|---|---|---|
| 0 | ติดตั้ง Node.js และ PostgreSQL · สร้างฐานข้อมูล | — |
| 1 | สร้างโปรเจกต์ · ติดตั้งแพ็กเกจ · ตั้งค่า TypeScript | `package.json` · `tsconfig.json` |
| 2 | ตั้งค่าการเชื่อมต่อ · เขียนพิมพ์เขียวฐานข้อมูล | `.env` · `prisma.config.ts` · `prisma/schema.prisma` |
| 3 | สร้างตารางจริง · สร้าง Prisma Client | `prisma/migrations/…/migration.sql` |
| 4 | ต่อฐานข้อมูล · ใส่ข้อมูลตัวอย่าง | `src/lib/prisma.ts` · `prisma/seed.ts` |
| 5 | เขียนยามที่ประตูหน้า (Zod) | `src/schemas/student.schema.ts` |
| 6 | เขียนชั้นจัดการข้อผิดพลาด | `src/errors/` · `src/middlewares/` (4 ไฟล์) |
| 7 | เขียนตรรกะ CRUD | `src/services/student.service.ts` |
| 8 | เขียนเส้นทาง HTTP | `src/routes/students.route.ts` |
| 9 | ประกอบร่าง · เปิดพอร์ต | `src/app.ts` · `src/server.ts` |
| 10 | รันและยิงให้ผ่านครบ 5 เส้น | — |
| 11 | ลองทำให้พังให้ครบทุกแบบ | — |

**จบขั้นที่ 11 แล้วโฟลเดอร์ของคุณคือ API ที่ต่อฐานข้อมูลจริงและป้องกันตัวเองได้**

เก็บไว้ให้ดี — สัปดาห์ถัดไปจะต่อยอดจากโฟลเดอร์นี้

# ขั้นที่ 0.1 — ติดตั้ง Node.js และตรวจว่าใช้ได้

### ถ้ายังไม่มี Node.js

1. เปิด **nodejs.org** → ดาวน์โหลดรุ่น **LTS** (ตัวเลขคู่ เช่น 20 หรือ 22)
2. ติดตั้งแบบกด Next ผ่านทั้งหมด (ไม่ต้องเปลี่ยนค่าใด ๆ)
3. **ปิดหน้าต่าง PowerShell/Terminal ที่เปิดอยู่ทั้งหมด แล้วเปิดใหม่** — ไม่ทำขั้นนี้จะหาคำสั่งไม่เจอ

### ตรวจว่าใช้ได้แล้ว

```
node --version    # ต้องได้ v20.x ขึ้นไป
npm --version     # ต้องได้ 10.x ขึ้นไป
```

| อาการ | สาเหตุและวิธีแก้ |
|---|---|
| `node : The term 'node' is not recognized` | ยังไม่ปิด-เปิดหน้าต่างใหม่ · หรือติดตั้งไม่สำเร็จ ให้ติดตั้งซ้ำ |
| เวอร์ชันเป็น v16 หรือต่ำกว่า | เก่าเกินไป — ติดตั้งรุ่น LTS ใหม่ทับได้เลย |
| `npm` ใช้ไม่ได้แต่ `node` ใช้ได้ | ติดตั้งไม่ครบ ให้ถอนแล้วติดตั้งใหม่ |

> **ห้ามใช้ Node.js รุ่นเลขคี่** (21, 23) — เป็นรุ่นทดลอง ไม่มีการดูแลระยะยาว

# ขั้นที่ 0.2 — ติดตั้ง PostgreSQL 17

1. เปิด **postgresql.org/download/windows** → กด *Download the installer*
2. เลือกรุ่น **17.x** สำหรับ Windows x86-64 → ดาวน์โหลด (ไฟล์ประมาณ 350 MB)
3. เปิดไฟล์ติดตั้ง แล้วทำตามนี้ทีละหน้า

| หน้าจอ | ต้องทำอะไร |
|---|---|
| *Select Components* | ติ๊ก **PostgreSQL Server** · **pgAdmin 4** · **Command Line Tools** (ทั้ง 3 ตัวจำเป็น) |
| *Password* | **ตั้งรหัสผ่านของผู้ใช้** `postgres` — จดไว้ให้ดี ลืมแล้วต้องติดตั้งใหม่ |
| *Port* | ปล่อยเป็น **5432** ตามค่าเริ่มต้น |
| *Locale* | ปล่อยเป็น **Default locale** |
| *Stack Builder* | **ไม่ต้องติ๊ก** — ปิดหน้านี้ไปได้เลย |

### ตรวจว่าบริการทำงานอยู่

```
Get-Service -Name "postgresql*"
# ต้องเห็น Status = Running
```

> **รหัสผ่านที่ตั้งในหน้า Password คือสิ่งที่จะใช้ในขั้นที่ 2** — เขียนใส่กระดาษไว้เลย
> ถ้าบริการไม่ Running ให้สั่ง `Start-Service postgresql-x64-17` ในหน้าต่างที่เปิดแบบ Run as Administrator

# ขั้นที่ 0.3 — สร้างฐานข้อมูลของคาบนี้

เราจะไม่ใช้ฐานข้อมูลตั้งต้น แต่สร้างฐานข้อมูลใหม่ชื่อ `ce385_week8` แยกไว้

### วิธีที่ 1 — ผ่าน pgAdmin 4 (แนะนำสำหรับคนที่เพิ่งเริ่ม)

1. เปิด **pgAdmin 4** → ใส่รหัสผ่านที่ตั้งไว้
2. แถบซ้าย คลิก *Servers* → *PostgreSQL 17* → คลิกขวาที่ **Databases** → *Create* → *Database…*
3. ช่อง *Database* พิมพ์ `ce385_week8` → กด **Save**

### วิธีที่ 2 — ผ่านบรรทัดคำสั่ง

```
# ถ้า psql หาไม่เจอ ให้ใช้ path เต็ม:
# & "C:\Program Files\PostgreSQL\17\bin\psql.exe" -U postgres -h localhost
psql -U postgres -h localhost -c "CREATE DATABASE ce385_week8;"
```

> **ข้อดีที่ไม่ค่อยมีใครบอก:** ขั้นนี้ข้ามได้ถ้าขี้เกียจ — `prisma migrate dev` ในขั้นที่ 3 จะสร้างฐานข้อมูลให้เองถ้ายังไม่มี แต่ **ควรทำเองครั้งแรก** เพื่อยืนยันว่ารหัสผ่านที่จดไว้ถูกจริง

# ขั้นที่ 0.4 — ทางหนีไฟถ้าติดตั้งไม่สำเร็จ

ถ้าลองแล้วแต่ยังติดตั้งลงเครื่องไม่ได้ **อย่าติดค้างอยู่ที่ขั้นนี้** — ใช้ PostgreSQL บนคลาวด์ฟรีไปก่อน

### Neon (ฟรี ไม่ต้องใส่บัตรเครดิต)

1. เปิด **neon.com** → *Sign up* ด้วยบัญชี Google หรือ GitHub
2. *Create project* → ตั้งชื่อ `ce385-week8` → เลือก Region ที่ใกล้ที่สุด (Singapore)
3. หน้าถัดไปจะมีกล่อง **Connection string** → กดปุ่มคัดลอก
4. เอาสายที่ได้ไปใส่ในไฟล์ `.env` ขั้นที่ 2 แทนสายแบบ localhost

```
postgresql://neondb_owner:xxxxxxxx@ep-cool-name-123456.ap-southeast-1.aws.neon.tech/neondb?sslmode=require
```

| ข้อดี | ข้อเสีย |
|---|---|
| ไม่ต้องติดตั้งอะไรเลย · ใช้ได้ใน 3 นาที | ต้องต่อเน็ตทุกครั้ง · ช้ากว่า localhost |
| เพื่อนในกลุ่มใช้ฐานข้อมูลเดียวกันได้ | ถ้าไม่ใช้นาน ๆ ฐานข้อมูลจะหลับ (คำขอแรกช้า 2–3 วินาที) |

> **แต่ต้องกลับมาติดตั้งลงเครื่องให้ได้ภายในสัปดาห์นี้** — สัปดาห์ที่ 12 เรียนเรื่อง Docker
> ซึ่งต้องเข้าใจว่าฐานข้อมูลบนเครื่องตัวเองทำงานอย่างไร

# ขั้นที่ 1.1 — สร้างโปรเจกต์เปล่า

```
# สร้างโฟลเดอร์และเข้าไปในนั้น
mkdir ce385-week8
cd ce385-week8

# สร้าง package.json แบบไม่ต้องตอบคำถาม
npm init -y
```

ตอนนี้จะมี `package.json` หนึ่งไฟล์ — เดี๋ยวเราจะแก้ทั้งไฟล์ในขั้นที่ 1.3

```
# สร้างโครงโฟลเดอร์ให้ครบตั้งแต่ตอนนี้ จะได้ไม่ต้องสร้างทีหลังทีละอัน
mkdir prisma
mkdir src
mkdir src\lib, src\schemas, src\services, src\routes, src\errors, src\middlewares
```

| โฟลเดอร์ | จะเก็บอะไร |
|---|---|
| `prisma/` | พิมพ์เขียวฐานข้อมูล · ประวัติ migration · สคริปต์ใส่ข้อมูลตัวอย่าง |
| `src/lib/` | ตัวเชื่อมต่อของภายนอกที่ใช้ร่วมกันทั้งโปรเจกต์ |
| `src/schemas/` | กติกาการตรวจข้อมูล (Zod) |
| `src/services/` | ตรรกะทางธุรกิจ — **ห้ามรู้จัก express** |
| `src/routes/` | เส้นทาง HTTP |
| `src/errors/` · `src/middlewares/` | ชั้นจัดการข้อผิดพลาดและงานที่ทุกคำขอต้องผ่าน |

# ขั้นที่ 1.2 — ติดตั้งแพ็กเกจ

```
# แพ็กเกจที่ต้องมีตอนรันจริง
npm install express zod @prisma/client @prisma/adapter-pg pg dotenv

# แพ็กเกจที่ใช้เฉพาะตอนพัฒนา
npm install -D typescript tsx prisma @types/node @types/express
```

| แพ็กเกจ | หน้าที่ | ทำไมต้องมี |
|---|---|---|
| `express` | เว็บเฟรมเวิร์ก | ของเดิมจากสัปดาห์ที่ 5 — **ปักรุ่น 4 ไว้** ให้ตรงกับที่เรียนมา |
| `zod` | ตรวจข้อมูล | ยามที่ประตูหน้า (ชั้นที่ 1) |
| `@prisma/client` | Prisma Client ตอนรัน | ตัวที่เรียก `prisma.student.findMany()` |
| `@prisma/adapter-pg` | หัวแปลงให้ Prisma คุยกับ `pg` | Prisma 7 บังคับให้ระบุ adapter |
| `pg` | ไดรเวอร์ PostgreSQL ของ Node.js | ตัวที่คุยกับฐานข้อมูลจริง ๆ |
| `dotenv` | อ่านไฟล์ `.env` | ไม่ต้องเอารหัสผ่านไปฝังในโค้ด |
| `typescript` | ตัวแปลภาษา | — |
| `tsx` | **รันไฟล์ `.ts` ได้เลยโดยไม่ต้อง build** | แก้ไฟล์แล้วรีสตาร์ทให้เอง เหมือน nodemon |
| `prisma` | เครื่องมือบรรทัดคำสั่ง | `migrate` · `generate` · `studio` |
| `@types/*` | คำอธิบายชนิดข้อมูล | ให้ TypeScript รู้จัก `express` และ API ของ Node.js |

# ขั้นที่ 1.3 — `package.json` (เต็มไฟล์)

```
{
  "name": "ce385-week8",
  "version": "1.0.0",
  "private": true,
  "type": "module",
  "scripts": {
    "dev": "tsx watch src/server.ts",
    "start": "node dist/server.js",
    "build": "tsc",
    "typecheck": "tsc --noEmit",
    "seed": "tsx prisma/seed.ts",
    "studio": "prisma studio"
  },
  "prisma": { "seed": "tsx prisma/seed.ts" },
  "dependencies": {
    "@prisma/adapter-pg": "^7.3.0",
    "@prisma/client": "^7.3.0",
    "dotenv": "^17.2.3",
    "express": "^4.21.2",
    "pg": "^8.17.2",
    "zod": "^4.1.12"
  },
  "devDependencies": {
    "@types/express": "^4.17.23",
    "@types/node": "^24.10.1",
    "prisma": "^7.3.0",
    "tsx": "^4.20.6",
    "typescript": "^5.9.3"
  }
}
```

`"type": "module"` คือบรรทัดที่สำคัญที่สุด — ทำให้ทั้งโปรเจกต์ใช้ `import` / `export`

# ขั้นที่ 1.4 — `tsconfig.json` (เต็มไฟล์)

```
{
  "compilerOptions": {
    "target": "ES2022",
    "module": "NodeNext",
    "moduleResolution": "NodeNext",
    "lib": ["ES2022"],

    "rootDir": "src",
    "outDir": "dist",
    "sourceMap": true,

    "strict": true,
    "noUncheckedIndexedAccess": true,
    "esModuleInterop": true,
    "forceConsistentCasingInFileNames": true,
    "skipLibCheck": true
  },
  "include": ["src/**/*.ts"],
  "exclude": ["node_modules", "dist"]
}
```

| ตัวเลือก | ทำอะไร |
|---|---|
| `module: NodeNext` | ใช้ ESM แบบเดียวกับที่ Node.js เข้าใจ — **ผลคือ `import` ต้องลงท้ายด้วย `.js`** |
| `strict: true` | เปิดการตรวจเข้มทั้งชุด · ห้าม `any` แฝง · ห้ามลืมเช็ก `null` |
| `noUncheckedIndexedAccess` | `arr[0]` มีชนิดเป็น `T \| undefined` — บังคับให้เช็กก่อนใช้ |
| `skipLibCheck: true` | ไม่ต้องตรวจไฟล์ชนิดข้อมูลของแพ็กเกจอื่น — คอมไพล์เร็วขึ้นมาก |

# ขั้นที่ 2.1 — `.env` และ `.env.example`

สร้างสองไฟล์ที่ **รากของโปรเจกต์** (ระดับเดียวกับ `package.json`)

### `.env.example` — ไฟล์ตัวอย่างที่ commit ขึ้น Git ได้

```
DATABASE_URL="postgresql://postgres:myPassword@localhost:5432/ce385_week8?schema=public"
PORT=3000
```

### `.env` — ไฟล์จริงของเครื่องตัวเอง ห้าม commit

```
Copy-Item .env.example .env
# แล้วเปิดแก้ myPassword เป็นรหัสผ่านจริงที่จดไว้ในขั้นที่ 0.2
```

### แกะสาย DATABASE_URL

```
postgresql:// postgres : myPassword @ localhost : 5432 / ce385_week8 ?schema=public
   โปรโตคอล     ผู้ใช้      รหัสผ่าน       โฮสต์       พอร์ต    ชื่อฐานข้อมูล     ชื่อ schema
```

**สร้างไฟล์ `.gitignore` ด้วย** ใส่บรรทัด `.env` · `node_modules` · `dist` · `src/generated`

**รหัสผ่านที่หลุดขึ้น GitHub ถือว่าหลุดไปแล้วตลอดกาล** แม้จะลบ commit ทิ้งภายหลัง

# ขั้นที่ 2.2 — `prisma.config.ts` (เต็มไฟล์)

```ts
// prisma.config.ts — ไฟล์ตั้งค่าของเครื่องมือ Prisma (Prisma 7 ขึ้นไป)
// เดิม (Prisma 6) เคยใส่ url ไว้ใน schema.prisma ตรง ๆ
// Prisma 7 ย้ายมาอยู่ที่นี่ เพราะไฟล์ .ts อ่าน process.env ได้
import "dotenv/config"; // โหลดค่าจาก .env เข้า process.env ก่อนใช้งาน
import { defineConfig, env } from "prisma/config";

export default defineConfig({
  schema: "prisma/schema.prisma",      // ไฟล์ที่บอกว่าฐานข้อมูลมีตารางอะไร
  migrations: {
    path: "prisma/migrations",         // ที่เก็บประวัติการเปลี่ยนแปลงฐานข้อมูล
    seed: "tsx prisma/seed.ts",        // คำสั่งที่ prisma db seed จะเรียก
  },
  datasource: {
    url: env("DATABASE_URL"),          // อ่านสายเชื่อมต่อจาก .env
  },
});
```

**ถ้าคุณเคยเห็นตัวอย่างในอินเทอร์เน็ตที่เขียน `url = env("DATABASE_URL")` ไว้ใน `schema.prisma`**
นั่นคือวิธีของ Prisma 6 — ยังใช้ได้ แต่ Prisma 7 ย้ายมาไว้ที่ไฟล์นี้แล้ว
อย่าใส่ทั้งสองที่ เพราะจะชนกันแล้วขึ้นข้อความเตือน

# ขั้นที่ 2.3 — `prisma/schema.prisma` (เต็มไฟล์)

```
generator client {
  provider     = "prisma-client"            // ตัวสร้างโค้ด Client รุ่นใหม่ของ Prisma 7
  output       = "../src/generated/prisma"  // โค้ดที่สร้างออกมาไปอยู่ที่นี่ (ห้ามแก้ด้วยมือ)
  moduleFormat = "esm"                      // ให้ออกมาเป็น ESM เพราะโปรเจกต์นี้ "type": "module"
}

datasource db {
  provider = "postgresql"   // เราใช้ PostgreSQL — สาย url อยู่ใน prisma.config.ts
}

/// นักศึกษา — ทรัพยากรเดียวของคาบนี้ ที่จะทำ CRUD ให้ครบ
model Student {
  id        String   @id @default(uuid()) @db.Uuid    // กุญแจหลัก ใช้ UUID ให้ Prisma สร้างเอง
  code      String   @unique @db.VarChar(10)          // รหัสนักศึกษา — @unique คือแนวป้องกันสุดท้าย
  name      String   @db.VarChar(100)                 // ชื่อ-นามสกุล จำกัด 100 ตัวอักษร
  email     String   @unique                          // อีเมล ห้ามซ้ำกับคนอื่น
  score     Int      @default(0)                      // คะแนน ถ้าไม่ส่งมาให้เป็น 0
  active    Boolean  @default(true)                   // ยังศึกษาอยู่หรือไม่
  createdAt DateTime @default(now())                  // เวลาที่สร้างแถวนี้ ฐานข้อมูลใส่ให้เอง
  updatedAt DateTime @updatedAt                       // เวลาที่แก้ครั้งล่าสุด Prisma อัปเดตให้เอง

  @@map("students")  // ชื่อตารางจริงเป็นพหูพจน์ตัวเล็ก ตามธรรมเนียม SQL
}
```

# ขั้นที่ 2.4 — อ่าน schema ให้เข้าใจทุกบรรทัด

| สัญลักษณ์ | ความหมาย | ผลที่เกิดใน PostgreSQL |
|---|---|---|
| `@id` | ฟิลด์นี้คือกุญแจหลัก | `PRIMARY KEY` |
| `@default(uuid())` | ถ้าไม่ส่งมา ให้สร้าง UUID ใหม่ | Prisma สร้างค่าให้ตอน `create` |
| `@db.Uuid` | เก็บเป็นชนิด `uuid` ของ PostgreSQL | ประหยัดที่และเทียบเร็วกว่าเก็บเป็นข้อความ |
| `@unique` | **ห้ามซ้ำทั้งตาราง** | `CREATE UNIQUE INDEX` — **นี่คือชั้นป้องกันที่ 3** |
| `@db.VarChar(100)` | ข้อความยาวไม่เกิน 100 | `VARCHAR(100)` |
| `@default(0)` | ค่าเริ่มต้นเป็น 0 | `DEFAULT 0` |
| `@updatedAt` | อัปเดตเวลาทุกครั้งที่แก้แถวนี้ | **Prisma จัดการให้ ไม่ใช่ฐานข้อมูล** |
| `String?` *(มี `?`)* | ยอมให้เป็นค่าว่างได้ | ไม่มี `NOT NULL` |
| `@@map("students")` | ชื่อโมเดลกับชื่อตารางต่างกันได้ | โค้ดใช้ `Student` · ตารางชื่อ `students` |

**`Int` เทียบกับ `Float` เทียบกับ `Decimal`:** คะแนนใช้ `Int` · น้ำหนักส่วนสูงใช้ `Float`
· **เรื่องเงินต้องใช้ `Decimal` เท่านั้น** เพราะ `Float` ปัดเศษเพี้ยน

# ขั้นที่ 3.1 — สร้างตารางจริงด้วย `migrate dev`

```
npx prisma migrate dev --name init
```

**สิ่งที่คำสั่งนี้ทำให้ 5 อย่างในครั้งเดียว**

1. อ่าน `schema.prisma` แล้วเทียบกับฐานข้อมูลว่าต่างกันตรงไหน
2. **เขียนไฟล์ SQL** ไว้ที่ `prisma/migrations/<เวลา>_init/migration.sql`
3. **รัน SQL นั้นกับฐานข้อมูลจริง** → ตาราง `students` เกิดขึ้น
4. จดไว้ในตาราง `_prisma_migrations` ว่า migration นี้ถูกใช้ไปแล้ว
5. สั่ง `prisma generate` **ให้เองอัตโนมัติ** → ได้ Prisma Client ตัวใหม่

**ผลที่ควรเห็นบนหน้าจอ**

```
PostgreSQL database ce385_week8 created at localhost:5432
Applying migration `20260927070235_init`
The following migration(s) have been created and applied from new schema changes:
prisma\migrations/
  └─ 20260927070235_init/
    └─ migration.sql
Your database is now in sync with your schema.
```

> `--name init` **คือชื่อที่เราตั้งให้การเปลี่ยนแปลงครั้งนี้** เหมือนข้อความ commit ของ Git
> ครั้งต่อไปที่เพิ่มฟิลด์ ให้ตั้งชื่อที่บอกได้ว่าทำอะไร เช่น `--name add_phone_to_student`

# ขั้นที่ 3.2 — เปิดดู SQL ที่ Prisma เขียนให้

เปิดไฟล์ `prisma/migrations/<เวลา>_init/migration.sql`

```sql
-- CreateTable
CREATE TABLE "students" (
    "id" UUID NOT NULL,
    "code" VARCHAR(10) NOT NULL,
    "name" VARCHAR(100) NOT NULL,
    "email" TEXT NOT NULL,
    "score" INTEGER NOT NULL DEFAULT 0,
    "active" BOOLEAN NOT NULL DEFAULT true,
    "createdAt" TIMESTAMP(3) NOT NULL DEFAULT CURRENT_TIMESTAMP,
    "updatedAt" TIMESTAMP(3) NOT NULL,

    CONSTRAINT "students_pkey" PRIMARY KEY ("id")
);

-- CreateIndex
CREATE UNIQUE INDEX "students_code_key" ON "students"("code");

-- CreateIndex
CREATE UNIQUE INDEX "students_email_key" ON "students"("email");
```

**หยุดอ่านไฟล์นี้ให้จบทุกบรรทัด** — นี่คือจุดที่เห็นชัดที่สุดว่า ORM ไม่ใช่เวทมนตร์

`@unique` สองตัวในโมเดล กลายเป็น `CREATE UNIQUE INDEX` สองบรรทัด

**สองบรรทัดนี้คือชั้นป้องกันที่ 3 ที่เราคุยกันในภาค ก**

# ขั้นที่ 3.3 — คำสั่ง Prisma ที่ต้องรู้จัก

| คำสั่ง | ทำอะไร | ใช้เมื่อไร |
|---|---|---|
| `npx prisma migrate dev --name <ชื่อ>` | สร้าง migration ใหม่ + รัน + generate | **ทุกครั้งที่แก้** `schema.prisma` |
| `npx prisma generate` | สร้าง Prisma Client ใหม่เท่านั้น | หลัง `git pull` ที่มีคนแก้ schema มา |
| `npx prisma migrate deploy` | รัน migration ที่มีอยู่ ไม่สร้างใหม่ | บนเซิร์ฟเวอร์จริงเท่านั้น |
| `npx prisma studio` | เปิดหน้าเว็บดู/แก้ข้อมูลในตาราง | เวลาอยากเห็นข้อมูลด้วยตา |
| `npx prisma migrate reset` | **ล้างฐานข้อมูลทั้งหมด** แล้วรัน migration ใหม่หมด | เวลาข้อมูลพังจนแก้ไม่ไหว |
| `npx prisma db seed` | รันสคริปต์ใส่ข้อมูลตัวอย่าง | หลัง reset หรือหลัง migrate ครั้งแรก |

### ลองเปิด Prisma Studio ดูตารางที่เพิ่งสร้าง

```
npx prisma studio      # จะเปิดเบราว์เซอร์ที่ http://localhost:5555
```

ตอนนี้ตาราง `students` มีอยู่แล้วแต่ **ยังว่างเปล่า** — ขั้นที่ 4.2 จะใส่ข้อมูลเข้าไป

> `migrate reset` **ลบข้อมูลทั้งหมดจริง ๆ** และไม่มีทางกู้คืน — ห้ามใช้กับฐานข้อมูลที่มีข้อมูลจริง

# ขั้นที่ 4.1 — `src/lib/prisma.ts` (เต็มไฟล์)

```ts
// src/lib/prisma.ts — จุดเดียวในโปรแกรมที่สร้าง PrismaClient
// ทำไมต้องมีไฟล์นี้: ถ้าใครอยากคุยกับฐานข้อมูลก็ import ตัวนี้ไปใช้
// จะได้มี connection pool ชุดเดียว ไม่ใช่เปิดใหม่ทุกไฟล์
import "dotenv/config";                          // โหลด .env ก่อนอ่าน process.env
import { PrismaPg } from "@prisma/adapter-pg";
import { PrismaClient } from "../generated/prisma/client.js";

const connectionString = process.env.DATABASE_URL;

if (!connectionString) {
  // ล้มให้ดังตั้งแต่ตอนเปิดโปรแกรม ดีกว่าไปพังตอนมีคนยิง request เข้ามา
  throw new Error("ไม่พบค่า DATABASE_URL — คัดลอก .env.example เป็น .env แล้วกรอกให้ครบก่อน");
}

// adapter คือ "หัวแปลง" ที่ให้ Prisma คุยกับไดรเวอร์ pg ของ PostgreSQL
const adapter = new PrismaPg({ connectionString });

export const prisma = new PrismaClient({ adapter });
```

**สังเกต `.js` ท้าย `../generated/prisma/client.js`** — ไฟล์จริงเป็น `.ts`
แต่ ESM อ้างถึงไฟล์ **ผลลัพธ์หลังคอมไพล์** เสมอ · นี่คือกฎที่นักศึกษาสะดุดมากที่สุดวันนี้
**ทุก import ในโปรเจกต์นี้ต้องลงท้ายด้วย `.js`** ยกเว้นการ import ชื่อแพ็กเกจ

# ขั้นที่ 4.2 — `prisma/seed.ts` (เต็มไฟล์)

```
// prisma/seed.ts — ใส่ข้อมูลตั้งต้นลงตารางว่าง
// รันด้วย: npx prisma db seed   (หรือ npm run seed)
import { prisma } from "../src/lib/prisma.js";

const students = [
  { code: "66010001", name: "สมชาย ใจดี",        email: "somchai@dpu.ac.th",   score: 78 },
  { code: "66010002", name: "สมหญิง รักเรียน",   email: "somying@dpu.ac.th",   score: 92 },
  { code: "66010003", name: "วีรภาพ ตั้งใจ",      email: "weeraphap@dpu.ac.th", score: 65 },
  { code: "66010004", name: "ปรียา ขยันทำ",       email: "preeya@dpu.ac.th",    score: 88 },
  { code: "66010005", name: "ธนกร ไม่ยอมแพ้",    email: "thanakorn@dpu.ac.th", score: 54, active: false },
];

async function main() {
  // ล้างของเก่าก่อน เพื่อให้รัน seed ซ้ำได้เรื่อย ๆ ผลลัพธ์เหมือนกันทุกครั้ง
  await prisma.student.deleteMany();

  // skipDuplicates: ถ้าเจอ code หรือ email ซ้ำให้ข้ามไป ไม่ต้องล้ม
  const result = await prisma.student.createMany({ data: students, skipDuplicates: true });

  console.log(`เพิ่มข้อมูลนักศึกษาแล้ว ${result.count} คน`);
}

main()
  .catch((error) => { console.error("seed ไม่สำเร็จ:", error); process.exit(1); })
  .finally(() => prisma.$disconnect());
```

```
npx prisma db seed     # ต้องได้ข้อความ: เพิ่มข้อมูลนักศึกษาแล้ว 5 คน
```

# ขั้นที่ 5.1 — `src/schemas/student.schema.ts` (ส่วนที่ 1/2)

```ts
// src/schemas/student.schema.ts — "ยาม" ที่ประตูหน้าของระบบ
// หน้าที่เดียว: บอกว่าข้อมูลที่ยอมรับได้ต้องมีรูปร่างแบบไหน
// ไฟล์นี้ไม่รู้จัก Express และไม่รู้จักฐานข้อมูล
import { z } from "zod";

/* ---------- 1) นิยามของแต่ละฟิลด์ เขียนครั้งเดียว ใช้ซ้ำได้ ---------- */
// ข้อความใน { error: "..." } คือข้อความที่ใช้เมื่อ "ชนิดข้อมูลผิด"
// ซึ่งรวมกรณีที่ไม่ส่งฟิลด์นี้มาเลยด้วย — ถ้าไม่ใส่ Zod จะขึ้นข้อความอังกฤษของมันเอง
const studentFields = {
  code: z
    .string({ error: "ต้องระบุรหัสนักศึกษา" })
    .trim()
    .regex(/^\d{8,10}$/, "รหัสนักศึกษาต้องเป็นตัวเลข 8-10 หลัก"),

  name: z
    .string({ error: "ต้องระบุชื่อ-นามสกุล" })
    .trim()
    .min(2, "ชื่อต้องยาวอย่างน้อย 2 ตัวอักษร")
    .max(100, "ชื่อยาวเกิน 100 ตัวอักษร"),

  email: z.email("รูปแบบอีเมลไม่ถูกต้อง"),

  score: z.coerce
    .number({ error: "คะแนนต้องเป็นตัวเลข" })
    .int("คะแนนต้องเป็นจำนวนเต็ม")
    .min(0, "คะแนนต้องไม่ต่ำกว่า 0")
    .max(100, "คะแนนต้องไม่เกิน 100"),

  active: z.boolean({ error: "active ต้องเป็น true หรือ false" }),
};
```

**เปรียบเทียบกับตารางในสไลด์ "ทรัพยากรของเราวันนี้"** — ทุกบรรทัดตรงกันหมด

# ขั้นที่ 5.2 — `src/schemas/student.schema.ts` (ส่วนที่ 2/2)

```
/* ---------- 2) POST /students — ต้องส่งมาให้ครบ ---------- */
// strictObject = ถ้าส่งฟิลด์ที่เราไม่รู้จักมาด้วย ให้ตีกลับ ไม่ใช่เมินเฉย
export const createStudentSchema = z.strictObject(
  {
    ...studentFields,
    score: studentFields.score.default(0),        // ไม่ส่งมา = 0
    active: studentFields.active.default(true),   // ไม่ส่งมา = true
  },
  { error: "ส่งฟิลด์ที่ระบบไม่รู้จักมาด้วย — ตรวจชื่อฟิลด์ให้ตรงกับเอกสาร" },
);

/* ---------- 3) PATCH /students/:id — ส่งมาแค่ฟิลด์ที่จะแก้ ---------- */
export const updateStudentSchema = z
  .strictObject(studentFields, {
    error: "ส่งฟิลด์ที่ระบบไม่รู้จักมาด้วย — ตรวจชื่อฟิลด์ให้ตรงกับเอกสาร",
  })
  .partial()                                       // ทุกฟิลด์เป็น "ส่งก็ได้ ไม่ส่งก็ได้"
  .refine((body) => Object.keys(body).length > 0, {
    error: "ต้องส่งอย่างน้อยหนึ่งฟิลด์ที่ต้องการแก้",
  });

/* ---------- 4) :id ใน URL — ต้องเป็น UUID ---------- */
export const studentIdParamSchema = z.strictObject({
  id: z.uuid("id ต้องเป็น UUID เช่น 3f0c1e8a-... (คัดลอกจากผลของ GET /students)"),
});

/* ---------- 5) Query string ของ GET /students ----------
   ค่าที่มาจาก URL เป็นข้อความล้วน ("2" ไม่ใช่ 2) จึงต้องใช้ z.coerce แปลงก่อนตรวจ */
export const listStudentsQuerySchema = z.object({
  page: z.coerce.number({ error: "page ต้องเป็นตัวเลข" }).int().min(1, "page ต้องเริ่มที่ 1").default(1),
  limit: z.coerce.number({ error: "limit ต้องเป็นตัวเลข" }).int().min(1, "limit ต้องไม่น้อยกว่า 1").max(100, "limit สูงสุด 100").default(10),
  q: z.string().trim().min(1).optional(),                              // คำค้นหา
  sort: z.enum(["code", "name", "score", "createdAt"]).default("createdAt"),
  order: z.enum(["asc", "desc"]).default("desc"),
});

/* ---------- 6) ให้ TypeScript ถอดชนิดข้อมูลจาก schema เอง ---------- */
export type CreateStudentInput = z.infer<typeof createStudentSchema>;
export type UpdateStudentInput = z.infer<typeof updateStudentSchema>;
export type ListStudentsQuery = z.infer<typeof listStudentsQuerySchema>;
```

# ขั้นที่ 5.3 — ความสามารถของ Zod ที่ใช้ในไฟล์นี้

| สิ่งที่ใช้ | ทำอะไร | ถ้าไม่ใช้จะเจออะไร |
|---|---|---|
| `z.strictObject` | ตีฟิลด์ที่ไม่รู้จักกลับ | ส่ง `{"scoer": 90}` มา ระบบเมินเฉยแล้วบันทึกโดยไม่มี `score` |
| `z.coerce.number()` | แปลงข้อความเป็นตัวเลขก่อนตรวจ | `?page=2` จะฟ้องว่า `"2"` ไม่ใช่ตัวเลข |
| `.default(0)` | ใส่ค่าให้เมื่อไม่ส่งมา | ต้องเขียน `?? 0` กระจายทั้งโปรเจกต์ |
| `.partial()` | ทำทุกฟิลด์ให้ไม่บังคับ | ต้องเขียน schema ของ PATCH ซ้ำอีกชุด |
| `.refine()` | กฎที่ต้องดูหลายฟิลด์ประกอบ | `PATCH {}` จะผ่านไปได้แล้วไปพังที่ Prisma |
| `.trim()` | ตัดช่องว่างหัวท้ายให้ | `" สมชาย "` กับ `"สมชาย"` กลายเป็นคนละคน |
| `z.enum([...])` | จำกัดค่าที่รับได้ | `?sort=<คอลัมน์ที่ไม่มี>` ทำให้ Prisma พัง 500 |
| `z.infer<...>` | **ถอดชนิดข้อมูล TypeScript จาก schema** | ต้องเขียน `interface` ซ้ำ แล้วมันจะไม่ตรงกันในที่สุด |

## `parse` เทียบกับ `safeParse`

```
schema.parse(data);      // ผิด → โยน ZodError ออกมาทันที (เหมาะกับใช้ในโค้ดที่มี try/catch ครอบ)
schema.safeParse(data);  // ผิด → คืน { success: false, error } ไม่โยนอะไร (เหมาะกับ middleware)
```

**`z.infer` คือเหตุผลที่คุ้มค่าที่สุดของการใช้ Zod กับ TypeScript**

# ขั้นที่ 6.1 — `src/errors/app-error.ts` (เต็มไฟล์)

```ts
// src/errors/app-error.ts — ข้อผิดพลาดที่ "เราตั้งใจให้เกิด"
// เอาไว้แยกจากข้อผิดพลาดที่ไม่คาดคิด (bug) ซึ่งต้องตอบ 500 เสมอ

export class AppError extends Error {
  /**
   * @param statusCode สถานะ HTTP ที่จะตอบกลับ เช่น 404
   * @param code       รหัสข้อผิดพลาดแบบตัวอักษรที่ฝั่ง client เอาไปเทียบได้ เช่น "NOT_FOUND"
   * @param message    ข้อความอธิบายให้มนุษย์อ่าน
   */
  constructor(
    public readonly statusCode: number,
    public readonly code: string,
    message: string,
  ) {
    super(message);
    this.name = "AppError";
  }
}

// ตัวช่วยสั้น ๆ ที่ใช้บ่อย เพื่อไม่ต้องพิมพ์ตัวเลขซ้ำทั้งโปรเจกต์
export const notFound = (message: string) => new AppError(404, "NOT_FOUND", message);
export const conflict = (message: string) => new AppError(409, "CONFLICT", message);
```

**ทำไมต้องมีคลาสของตัวเอง** — เพราะตัวดักรวมศูนย์ต้องแยกให้ออกระหว่าง
"ข้อผิดพลาดที่เราวางแผนไว้แล้ว" (ตอบ status ตามที่ระบุ) กับ
"ข้อผิดพลาดที่ไม่คาดคิด" (ต้องตอบ 500 และไม่บอกรายละเอียดออกไป)
ถ้าใช้ `throw new Error("not found")` เฉย ๆ จะแยกไม่ออก และทุกอย่างจะกลายเป็น 500

# ขั้นที่ 6.2 — `src/middlewares/validate.ts` (เต็มไฟล์)

```ts
// src/middlewares/validate.ts — ตัวตรวจข้อมูลรวมศูนย์
// แนวคิด: route ไม่ต้องเขียน if ตรวจข้อมูลเองแม้แต่บรรทัดเดียว
import type { RequestHandler } from "express";
import type { ZodError, ZodType } from "zod";

interface Schemas {
  params?: ZodType;  // ส่วนที่อยู่ใน path เช่น /:id
  query?: ZodType;   // ส่วนที่อยู่หลัง ? เช่น ?page=2
  body?: ZodType;    // ส่วนที่อยู่ในตัวคำขอ (JSON)
}

// ผิดที่ path หรือ query = "คำขอเขียนมาผิดรูป"        → 400 Bad Request
// ผิดที่ body           = "รูปแบบถูก แต่เนื้อหาใช้ไม่ได้" → 422 Unprocessable Entity
const STATUS_OF: Record<keyof Schemas, number> = { params: 400, query: 400, body: 422 };

export function validate(schemas: Schemas): RequestHandler {
  return (req, res, next) => {
    const valid: Record<string, unknown> = {};

    // ตรวจตามลำดับ params → query → body
    // เจอผิดที่แรกแล้วหยุดทันที ไม่ต้องตรวจต่อ — นี่คือ fail-fast
    for (const source of ["params", "query", "body"] as const) {
      const schema = schemas[source];
      if (!schema) continue;

      const result = schema.safeParse(req[source]);

      if (!result.success) {
        // ติดป้ายไว้ว่าผิดที่ส่วนไหน เพื่อให้ตัวดักรวมศูนย์เลือก status code ได้ถูก
        (result.error as ZodError & { httpStatus?: number }).httpStatus = STATUS_OF[source];
        next(result.error);   // ไม่ตอบเองที่นี่ — ส่งต่อให้ error-handler
        return;
      }

      valid[source] = result.data;   // ได้ข้อมูลที่แปลงชนิดเรียบร้อยแล้ว
    }

    res.locals.valid = valid;
    next();   // ผ่านด่านแล้ว ส่งต่อให้ handler ตัวถัดไป
  };
}
```

# ขั้นที่ 6.2 (ต่อ) — ทำไมต้องใช้ `res.locals`

## ปัญหาที่ต้องหลบ

```
req.query = result.data;   // ❌ ใน Express 4 req.query เป็น getter — เขียนทับไม่ได้
                           //    ใน ESM จะ throw ออกมาเลย ไม่ได้เงียบ ๆ
```

## ทางออกที่ Express เตรียมไว้ให้แล้ว

`res.locals` คือ **ออบเจกต์ว่างที่เกิดใหม่ทุกคำขอ และตายไปพร้อมคำขอนั้น**

เหมือน "กระเป๋าติดตัว" ที่ middleware ใส่ของไว้ให้ handler ข้างหลังหยิบใช้

```
// ใน validate.ts (ผู้ใส่ของ)
res.locals.valid = { params: {...}, query: {...}, body: {...} };

// ใน route handler (ผู้หยิบของ)
const input: CreateStudentInput = res.locals.valid.body;
```

| ที่เก็บ | อายุ | เหมาะกับ |
|---|---|---|
| ตัวแปรระดับโมดูล | ตลอดอายุโปรแกรม | ค่าตั้งต้นที่ไม่เปลี่ยน |
| `res.locals` | **คำขอใบเดียว** | **ข้อมูลที่ผ่านการตรวจ · ผู้ใช้ที่ล็อกอิน · request id** |
| ฐานข้อมูล | ถาวร | ข้อมูลจริงของระบบ |

# ขั้นที่ 6.3 — `async-handler.ts` และ `logger.ts` (เต็มทั้งสองไฟล์)

```ts
// src/middlewares/async-handler.ts — ของเดิมจากสัปดาห์ที่ 5
// Express 4 ไม่รู้จัก Promise ที่ reject ข้างใน handler
// ถ้าไม่ห่อด้วยตัวนี้ คำขอจะค้างจนหมดเวลา ไม่มี error ออกมาให้เห็น
// (หมายเหตุ: Express 5 ดักให้เองแล้ว ไม่ต้องใช้ไฟล์นี้)
import type { RequestHandler } from "express";

export function asyncHandler(handler: RequestHandler): RequestHandler {
  return (req, res, next) => {
    Promise.resolve(handler(req, res, next)).catch(next);
  };
}
```

```ts
// src/middlewares/logger.ts — บันทึกทุกคำขอลง console
// มีไว้เพื่อ "เห็นด้วยตา" ว่าคำขอเข้ามาจริง และจบด้วย status อะไร
import type { RequestHandler } from "express";

export const logger: RequestHandler = (req, res, next) => {
  const startedAt = Date.now();

  // res.on("finish") ยิงเมื่อ Express ส่ง response ออกไปเรียบร้อยแล้ว
  res.on("finish", () => {
    const ms = Date.now() - startedAt;
    console.log(`${req.method} ${req.originalUrl} → ${res.statusCode} (${ms} ms)`);
  });

  next();
};
```

`logger` **ต้องอ่าน** `res.statusCode` **ตอนคำขอจบ ไม่ใช่ตอนเริ่ม** — ตอนเริ่มยังไม่มีใครตั้งค่าไว้ จึงต้องดักเหตุการณ์ `finish` แทนการพิมพ์ตรง ๆ

# ขั้นที่ 6.4 — `src/middlewares/error-handler.ts` (ส่วนที่ 1/2)

```ts
// src/middlewares/error-handler.ts — ตัวดักข้อผิดพลาดรวมศูนย์
// กฎเหล็ก: ในระบบนี้มี "ที่เดียว" ที่แปลง error เป็น HTTP response
// ทุกที่ที่เหลือมีหน้าที่แค่ next(error) หรือ throw ออกมา
// ต้องมี 4 พารามิเตอร์ (err, req, res, next) Express จึงจะรู้ว่าเป็นตัวดัก error
import type { ErrorRequestHandler, RequestHandler } from "express";
import { ZodError } from "zod";
import { Prisma } from "../generated/prisma/client.js";
import { AppError } from "../errors/app-error.js";

/** รูปแบบข้อผิดพลาดชุดเดียวที่ใช้ทั้งระบบ — ฝั่ง client แกะที่เดียวจบ */
interface ErrorBody {
  error: { code: string; message: string; details?: unknown };
}

/* ---------- ไม่มี route ไหนรับคำขอนี้ ---------- */
export const notFoundHandler: RequestHandler = (req, res) => {
  const body: ErrorBody = {
    error: {
      code: "ROUTE_NOT_FOUND",
      message: `ไม่มีเส้นทาง ${req.method} ${req.originalUrl} ในระบบนี้`,
    },
  };
  res.status(404).json(body);
};

/* ---------- ตัวดักหลัก ---------- */
export const errorHandler: ErrorRequestHandler = (err, _req, res, _next) => {
  // 1) ข้อมูลที่ส่งมาไม่ผ่าน schema
  //    400 ถ้าผิดที่ path/query · 422 ถ้าผิดที่ body (validate() ติดป้าย httpStatus มาให้)
  if (err instanceof ZodError) {
    const status = (err as ZodError & { httpStatus?: number }).httpStatus ?? 422;
    const body: ErrorBody = {
      error: {
        code: "VALIDATION_ERROR",
        message: "ข้อมูลที่ส่งมาไม่ถูกต้อง",
        details: err.issues.map((issue) => ({
          field: issue.path.join(".") || "(ทั้งก้อน)",
          message: issue.message,
        })),
      },
    };
    res.status(status).json(body);
    return;
  }

  // 2) ข้อผิดพลาดที่เราตั้งใจโยนเอง → ใช้ status ที่ระบุไว้
  if (err instanceof AppError) {
    const body: ErrorBody = { error: { code: err.code, message: err.message } };
    res.status(err.statusCode).json(body);
    return;
  }
  // ...ต่อในสไลด์ถัดไป
```

# ขั้นที่ 6.5 — `src/middlewares/error-handler.ts` (ส่วนที่ 2/2)

```
  // 3) ข้อผิดพลาดจากฐานข้อมูล — Prisma มีรหัสของตัวเอง แปลงเป็น HTTP ให้ถูกความหมาย
  if (err instanceof Prisma.PrismaClientKnownRequestError) {
    const map: Record<string, { status: number; code: string; message: string }> = {
      // ชนกับ @unique — เช่นรหัสนักศึกษาหรืออีเมลซ้ำ
      P2002: { status: 409, code: "DUPLICATE",      message: "ข้อมูลนี้มีอยู่แล้วในระบบ" },
      // update / delete แถวที่ไม่มีอยู่
      P2025: { status: 404, code: "NOT_FOUND",      message: "ไม่พบข้อมูลที่ต้องการแก้ไขหรือลบ" },
      // เชื่อมต่อฐานข้อมูลไม่ได้
      P1001: { status: 503, code: "DB_UNAVAILABLE", message: "ติดต่อฐานข้อมูลไม่ได้" },
    };

    const hit = map[err.code];
    if (hit) {
      const target = (err.meta as { target?: string[] } | undefined)?.target;
      const body: ErrorBody = {
        error: {
          code: hit.code,
          message: hit.message,
          details: target ? { field: target.join(", ") } : undefined,
        },
      };
      res.status(hit.status).json(body);
      return;
    }
  }

  // 4) ที่เหลือคือสิ่งที่เราไม่ได้คาดไว้ = bug ของเรา
  //    ตอบ 500 แบบกลาง ๆ ไม่รั่วรายละเอียดภายในออกไปให้ client
  //    แต่พิมพ์ลง console ให้ครบ เพื่อให้ผู้พัฒนาตามแก้ได้
  console.error("[UNHANDLED]", err);
  const body: ErrorBody = {
    error: { code: "INTERNAL_ERROR", message: "เกิดข้อผิดพลาดภายในระบบ" },
  };
  res.status(500).json(body);
};
```

**ลำดับของ `if` สำคัญ** — ตรวจจากเฉพาะเจาะจงที่สุดไปกว้างที่สุด

# ขั้นที่ 6.5 (ต่อ) — ตารางแปลงข้อผิดพลาด

| ต้นทาง | เงื่อนไข | ตอบ | `code` ที่ client เห็น |
|---|---|---|---|
| Zod | ผิดที่ `params` หรือ `query` | **400** | `VALIDATION_ERROR` |
| Zod | ผิดที่ `body` | **422** | `VALIDATION_ERROR` |
| `AppError` | `notFound()` | **404** | `NOT_FOUND` |
| `AppError` | `conflict()` | **409** | `CONFLICT` |
| Prisma | `P2002` ชนกับ `@unique` | **409** | `DUPLICATE` |
| Prisma | `P2025` ไม่พบแถวที่จะแก้/ลบ | **404** | `NOT_FOUND` |
| Prisma | `P1001` ต่อฐานข้อมูลไม่ได้ | **503** | `DB_UNAVAILABLE` |
| ไม่มี route รับ | — | **404** | `ROUTE_NOT_FOUND` |
| อย่างอื่นทั้งหมด | — | **500** | `INTERNAL_ERROR` |

## รหัสข้อผิดพลาดของ Prisma ที่เจอบ่อยอีก 3 ตัว

| รหัส | หมายความว่า | ควรตอบ |
|---|---|---|
| `P2003` | ผิดข้อจำกัด foreign key | 409 หรือ 422 |
| `P2000` | ค่ายาวเกินความกว้างของคอลัมน์ | 422 |

# ขั้นที่ 7.1 — `student.service.ts` (ส่วนที่ 1/2 — อ่านข้อมูล)

```ts
// src/services/student.service.ts — ชั้นตรรกะทางธุรกิจ
// กฎของชั้นนี้ (บังคับใช้ตั้งแต่สัปดาห์ที่ 5):
//   • ห้าม import express — ไม่รู้จัก req / res / status code
//   • ห้าม console.log — ใครเรียกไปใช้ก็ต้องได้ผลเหมือนกันหมด
import { prisma } from "../lib/prisma.js";
import { notFound } from "../errors/app-error.js";
import type { CreateStudentInput, ListStudentsQuery, UpdateStudentInput } from "../schemas/student.schema.js";

/* ---------- R: อ่านหลายรายการ + แบ่งหน้า + ค้นหา + เรียง ---------- */
export async function listStudents(query: ListStudentsQuery) {
  const { page, limit, q, sort, order } = query;

  // where ว่าง = ไม่กรองอะไร · mode: "insensitive" = ไม่สนตัวพิมพ์เล็กใหญ่
  const where = q
    ? {
        OR: [
          { name: { contains: q, mode: "insensitive" as const } },
          { email: { contains: q, mode: "insensitive" as const } },
          { code: { contains: q } },
        ],
      }
    : {};

  // ยิงสองคำถามพร้อมกัน — Promise.all จากสัปดาห์ที่ 4 ได้ใช้จริงที่นี่
  const [rows, total] = await Promise.all([
    prisma.student.findMany({
      where,
      orderBy: { [sort]: order },
      skip: (page - 1) * limit,   // ข้ามของหน้าก่อน ๆ
      take: limit,                // เอามาแค่เท่าที่ขอ
    }),
    prisma.student.count({ where }),
  ]);

  return { data: rows, meta: { page, limit, total, totalPages: Math.ceil(total / limit) } };
}

/* ---------- R: อ่านรายการเดียว ---------- */
export async function getStudentById(id: string) {
  const student = await prisma.student.findUnique({ where: { id } });

  // findUnique ไม่โยน error เมื่อหาไม่เจอ — มันคืน null เฉย ๆ
  // เราจึงต้องแปลง null เป็นข้อผิดพลาดที่มีความหมายเอง
  if (!student) throw notFound(`ไม่พบนักศึกษา id ${id}`);

  return student;
}
```

# ขั้นที่ 7.2 — `student.service.ts` (ส่วนที่ 2/2 — เขียนข้อมูล)

```ts
/* ---------- C: สร้างใหม่ ---------- */
export async function createStudent(input: CreateStudentInput) {
  // ถ้า code หรือ email ซ้ำ Prisma จะโยน P2002 ออกมา
  // เราปล่อยให้มันลอยขึ้นไปถึง error-handler ซึ่งแปลงเป็น 409 ให้เอง
  return prisma.student.create({ data: input });
}

/* ---------- U: แก้บางฟิลด์ ---------- */
export async function updateStudent(id: string, input: UpdateStudentInput) {
  // แถวไม่มีอยู่ → Prisma โยน P2025 → error-handler แปลงเป็น 404
  return prisma.student.update({ where: { id }, data: input });
}

/* ---------- D: ลบ ---------- */
export async function deleteStudent(id: string) {
  await prisma.student.delete({ where: { id } });
}
```

## สังเกตความต่างของสองวิธีจัดการ "ไม่พบข้อมูล"

| ฟังก์ชัน | วิธีที่ใช้ | เหตุผล |
|---|---|---|
| `getStudentById` | เช็ก `null` แล้ว `throw notFound(...)` เอง | `findUnique` คืน `null` ไม่โยน error · และเราอยากใส่ `id` ลงในข้อความ |
| `updateStudent` · `deleteStudent` | ปล่อยให้ Prisma โยน `P2025` เอง | ประหยัดการยิง query เพิ่มหนึ่งครั้ง และผลลัพธ์ที่ client เห็นเหมือนกัน |

**ทำไม service มีแค่ 5 ฟังก์ชันสั้น ๆ** — เพราะ Prisma ทำงานหนักไปแล้วทั้งหมด
**นี่คือประโยชน์ของ ORM ที่วัดได้:** โค้ดที่เราต้องดูแลลดลง แต่ความปลอดภัยของชนิดข้อมูลเพิ่มขึ้น

# ขั้นที่ 8.1 — `students.route.ts` (ส่วนที่ 1/2 — อ่าน)

```ts
// src/routes/students.route.ts — ชั้นเปลือกนอกสุดที่ติดกับ HTTP
// หน้าที่มีแค่ 3 อย่าง:
//   1. บอกว่า method ไหน + path ไหน
//   2. บอกว่าใช้ schema ไหนตรวจ
//   3. เอาผลจาก service มาห่อเป็น response พร้อม status code ที่ถูกต้อง
// ไม่มีตรรกะทางธุรกิจในไฟล์นี้ และ "ไม่มี /api/v1/students" ในไฟล์นี้ด้วย
// (path ที่เอาไปแขวนเป็นเรื่องของ app.ts — router ไม่ควรรู้ว่าตัวเองอยู่ที่ไหน)
import { Router } from "express";
import { asyncHandler } from "../middlewares/async-handler.js";
import { validate } from "../middlewares/validate.js";
import {
  createStudentSchema, listStudentsQuerySchema, studentIdParamSchema, updateStudentSchema,
  type CreateStudentInput, type ListStudentsQuery, type UpdateStudentInput,
} from "../schemas/student.schema.js";
import * as studentService from "../services/student.service.js";

export const studentsRouter = Router();

/* ---------- GET / → อ่านหลายรายการ ---------- */
studentsRouter.get(
  "/",
  validate({ query: listStudentsQuerySchema }),
  asyncHandler(async (_req, res) => {
    const query: ListStudentsQuery = res.locals.valid.query;
    const result = await studentService.listStudents(query);
    res.status(200).json(result);            // 200 OK + { data, meta }
  }),
);

/* ---------- GET /:id → อ่านรายการเดียว ---------- */
studentsRouter.get(
  "/:id",
  validate({ params: studentIdParamSchema }),
  asyncHandler(async (_req, res) => {
    const { id } = res.locals.valid.params as { id: string };
    const student = await studentService.getStudentById(id);
    res.status(200).json({ data: student });
  }),
);
```

**สังเกตลำดับ:** `validate(...)` มาก่อน `asyncHandler(...)` เสมอ

# ขั้นที่ 8.2 — `students.route.ts` (ส่วนที่ 2/2 — เขียน)

```ts
/* ---------- POST / → สร้างใหม่ ---------- */
studentsRouter.post(
  "/",
  validate({ body: createStudentSchema }),
  asyncHandler(async (_req, res) => {
    const input: CreateStudentInput = res.locals.valid.body;
    const created = await studentService.createStudent(input);

    // 201 Created + header Location ชี้ไปที่ของใหม่ที่เพิ่งสร้าง
    // นี่คือธรรมเนียมของ REST ที่คนลืมบ่อยที่สุด
    res.status(201).location(`/api/v1/students/${created.id}`).json({ data: created });
  }),
);

/* ---------- PATCH /:id → แก้บางฟิลด์ ---------- */
studentsRouter.patch(
  "/:id",
  validate({ params: studentIdParamSchema, body: updateStudentSchema }),
  asyncHandler(async (_req, res) => {
    const { id } = res.locals.valid.params as { id: string };
    const input: UpdateStudentInput = res.locals.valid.body;
    const updated = await studentService.updateStudent(id, input);
    res.status(200).json({ data: updated });
  }),
);

/* ---------- DELETE /:id → ลบ ---------- */
studentsRouter.delete(
  "/:id",
  validate({ params: studentIdParamSchema }),
  asyncHandler(async (_req, res) => {
    const { id } = res.locals.valid.params as { id: string };
    await studentService.deleteStudent(id);

    // 204 No Content — ลบสำเร็จแล้วไม่มีอะไรจะส่งกลับ
    // ห้ามใส่ body ใน 204 (ผิดมาตรฐาน HTTP) จึงใช้ .end() ไม่ใช่ .json()
    res.status(204).end();
  }),
);
```

**`PATCH` ตรวจสองส่วนพร้อมกัน** — `params` (400 ถ้าผิด) และ `body` (422 ถ้าผิด)

# ขั้นที่ 9.1 — `src/app.ts` (เต็มไฟล์)

```ts
// src/app.ts — ที่ประกอบร่าง
// ไฟล์นี้ตอบคำถามเดียว: "คำขอหนึ่งใบเดินผ่านอะไรบ้าง ตามลำดับไหน"
// ไม่มี app.listen ที่นี่ เพื่อให้เอา app ไปใช้กับการทดสอบได้ด้วย
import express from "express";
import { logger } from "./middlewares/logger.js";
import { errorHandler, notFoundHandler } from "./middlewares/error-handler.js";
import { studentsRouter } from "./routes/students.route.js";

export const app = express();

/* ---------- 1) middleware ที่ทุกคำขอต้องผ่าน ---------- */
app.use(express.json());   // แปลง JSON ใน body ให้เป็น object · ลืมบรรทัดนี้ req.body จะเป็น undefined
app.use(logger);           // บันทึกทุกคำขอ

/* ---------- 2) เส้นสำหรับเช็กว่าเซิร์ฟเวอร์ยังมีชีวิต ---------- */
app.get("/health", (_req, res) => {
  res.json({ status: "ok" });
});

/* ---------- 3) ทรัพยากรของเรา ---------- */
// ใส่เวอร์ชันไว้ใน path ตั้งแต่วันแรก — วันที่ต้องแก้สัญญาแบบไม่เข้ากันกับของเดิม
// จะได้เปิด /api/v2 ขึ้นมาขนานกันได้ โดยไม่ทำให้ของเก่าพัง
app.use("/api/v1/students", studentsRouter);

/* ---------- 4) สองตัวนี้ต้องอยู่ท้ายสุดเสมอ ---------- */
app.use(notFoundHandler);   // ไม่มี route ไหนรับ → 404
app.use(errorHandler);      // มี error โผล่มาจากที่ไหนก็ตาม → มาจบที่นี่
```

**ไฟล์นี้อ่านจากบนลงล่างได้เหมือนอ่านแผนผัง** — ลำดับในโค้ดคือลำดับที่คำขอเดินผ่านจริง
**ย้าย `errorHandler` ขึ้นไปไว้ก่อน `app.use("/api/v1/students", ...)` แล้วทั้งระบบจะพัง**
เพราะ error ที่เกิดหลังจากนั้นจะไม่มีใครดัก

# ขั้นที่ 9.2 — `src/server.ts` (เต็มไฟล์)

```ts
// src/server.ts — จุดเริ่มโปรแกรม
// หน้าที่เดียว: เปิดพอร์ต
// แยกจาก app.ts เพราะการ "ประกอบร่าง" กับการ "เปิดพอร์ต" คือสองเรื่อง
import "dotenv/config";
import { app } from "./app.js";
import { prisma } from "./lib/prisma.js";

const PORT = Number(process.env.PORT ?? 3000);

const server = app.listen(PORT, () => {
  console.log(`เซิร์ฟเวอร์พร้อมรับคำขอที่ http://localhost:${PORT}`);
  console.log(`ลองยิงดู: curl http://localhost:${PORT}/api/v1/students`);
});

/* ---------- ปิดโปรแกรมให้เรียบร้อย ---------- */
// กด Ctrl+C แล้วต้องคืน connection ให้ฐานข้อมูลก่อนตาย
// ไม่ทำก็ไม่พังทันที แต่ถ้ารีสตาร์ทบ่อย ๆ connection จะค้างจนเต็ม pool
const shutdown = async () => {
  console.log("\nกำลังปิดเซิร์ฟเวอร์...");
  server.close();
  await prisma.$disconnect();
  process.exit(0);
};

process.on("SIGINT", shutdown);
process.on("SIGTERM", shutdown);
```

**`prisma.$disconnect()` สำคัญกับงานจริงมาก** — PostgreSQL ยอมให้มี connection พร้อมกันได้จำนวนจำกัด (ค่าเริ่มต้น 100) · ถ้าเราไม่คืน connection ทุกครั้งที่ปิดโปรแกรม พัฒนาไปสักพักจะเจอ `too many clients already` แล้วต้องรีสตาร์ท PostgreSQL ทั้งตัว

# ขั้นที่ 10.1 — รันเซิร์ฟเวอร์

```
npm run dev
```

**ผลที่ต้องเห็น**

```
> ce385-week8@1.0.0 dev
> tsx watch src/server.ts

เซิร์ฟเวอร์พร้อมรับคำขอที่ http://localhost:3000
ลองยิงดู: curl http://localhost:3000/api/v1/students
```

| ถ้าเจออาการนี้ | สาเหตุ | วิธีแก้ |
|---|---|---|
| `EADDRINUSE: address already in use :::3000` | มีโปรแกรมอื่นใช้พอร์ต 3000 อยู่ | ปิดหน้าต่างที่รันค้างไว้ · หรือแก้ `PORT=3001` ใน `.env` |
| `ไม่พบค่า DATABASE_URL` | ยังไม่ได้สร้างไฟล์ `.env` | `Copy-Item .env.example .env` แล้วแก้รหัสผ่าน |
| `Cannot find module '../generated/prisma/client.js'` | ยังไม่ได้ generate | `npx prisma generate` |
| `Cannot find module './app'` (ไม่มี `.js`) | ลืมใส่ `.js` ท้าย import | เติม `.js` ให้ครบทุก import ในโปรเจกต์ |
| `P1001: Can't reach database server` | PostgreSQL ไม่ได้รัน หรือรหัสผ่านผิด | `Get-Service postgresql*` · ตรวจ `.env` อีกครั้ง |

`tsx watch` **เฝ้าดูไฟล์อยู่** — แก้ไฟล์แล้วบันทึก มันจะรีสตาร์ทให้เอง
**เปิดหน้าต่างที่สองไว้ยิงคำสั่งทดสอบ** ห้ามปิดหน้าต่างที่รันเซิร์ฟเวอร์

# ขั้นที่ 10.2 — ยิงให้ครบทั้ง 5 เส้น

เปิด **หน้าต่างที่สอง** แล้วยิงตามลำดับนี้

```
# 1) อ่านทั้งหมด (แบ่งหน้า) — ต้องได้ 200 และเห็นนักศึกษา 5 คน
curl "http://localhost:3000/api/v1/students?page=1&limit=3"

# 2) ค้นหา + เรียง
curl "http://localhost:3000/api/v1/students?q=somying&sort=score&order=asc"

# 3) สร้างใหม่ — ต้องได้ 201
curl -i -X POST http://localhost:3000/api/v1/students `
  -H "Content-Type: application/json" `
  -d '{\"code\":\"66019999\",\"name\":\"Test Student\",\"email\":\"test@dpu.ac.th\",\"score\":70}'

# คัดลอก id ที่ได้จากข้อ 3 มาใส่แทน <ID> ในสามข้อถัดไป

# 4) อ่านรายการเดียว — ต้องได้ 200
curl http://localhost:3000/api/v1/students/<ID>

# 5) แก้คะแนน — ต้องได้ 200
curl -X PATCH http://localhost:3000/api/v1/students/<ID> `
  -H "Content-Type: application/json" -d '{\"score\":95}'

# 6) ลบ — ต้องได้ 204 และไม่มีเนื้อหาตอบกลับ
curl -i -X DELETE http://localhost:3000/api/v1/students/<ID>
```

`-i` **ทำให้เห็น header ด้วย** — ใช้กับข้อ 3 เพื่อดู `Location:` และกับข้อ 6 เพื่อยืนยันว่าเป็น `204`

**PowerShell ต้อง escape เครื่องหมายคำพูดใน JSON ด้วย** `\"` · ถ้าใช้ Git Bash ไม่ต้อง escape

**ถ้าขี้เกียจ escape ให้ใช้ Postman** — สร้าง request 5 อันแล้วกดยิงได้เลย

# ขั้นที่ 10.3 — ตารางผลที่ต้องได้ (ตรวจให้ครบทุกช่อง)

| # | คำขอ | Status | สิ่งที่ต้องเห็นในคำตอบ |
|---|---|---|---|
| 1 | `GET /students?page=1&limit=3` | **200** | `data` มี 3 คน · `meta` = `{page:1, limit:3, total:5, totalPages:2}` |
| 2 | `GET /students?q=somying` | **200** | `data` มี 1 คน · `total: 1` |
| 3 | `POST` ข้อมูลถูกต้อง | **201** | header `Location: /api/v1/students/<id>` · `data.score` = 70 |
| 3b | `POST` ไม่ส่ง `score` มา | **201** | `data.score` = **0** · `data.active` = **true** (มาจาก `.default()`) |
| 4 | `GET /students/<id>` | **200** | `data` เป็นคนเดียวกับที่เพิ่งสร้าง |
| 5 | `PATCH {"score":95}` | **200** | `data.score` = 95 · **`updatedAt` เปลี่ยนไปจากเดิม** |
| 6 | `DELETE /students/<id>` | **204** | ไม่มีเนื้อหาตอบกลับเลย (`Content-Length: 0`) |
| 7 | `GET /health` | **200** | `{"status":"ok"}` |

**และในหน้าต่างที่รันเซิร์ฟเวอร์ต้องเห็น log ไหลออกมา**

```
GET /api/v1/students?page=1&limit=3 → 200 (18 ms)
POST /api/v1/students → 201 (24 ms)
PATCH /api/v1/students/309a768a-... → 200 (11 ms)
DELETE /api/v1/students/309a768a-... → 204 (9 ms)
```

**ถ้าถึงตรงนี้ผ่านครบ 8 ช่อง — CRUD ของคุณใช้งานได้จริงบนฐานข้อมูลจริงแล้ว**

เปิด `npx prisma studio` ดูตารางประกอบได้ จะเห็นแถวเปลี่ยนไปตามที่ยิง

# ขั้นที่ 11.1 — ลองทำให้พัง (ชั้นที่ 1 — Zod)

**ขั้นนี้สำคัญเท่ากับขั้นที่ 10** — API ที่ดีต้องปฏิเสธข้อมูลผิดได้อย่างชัดเจน

```
# ก) ไม่ส่งอะไรมาเลย → 422 และต้องบอกครบทั้ง 3 ฟิลด์ที่ขาด
curl -X POST http://localhost:3000/api/v1/students -H "Content-Type: application/json" -d '{}'
```

```
{"error":{"code":"VALIDATION_ERROR","message":"ข้อมูลที่ส่งมาไม่ถูกต้อง","details":[
  {"field":"code","message":"ต้องระบุรหัสนักศึกษา"},
  {"field":"name","message":"ต้องระบุชื่อ-นามสกุล"},
  {"field":"email","message":"รูปแบบอีเมลไม่ถูกต้อง"}]}}
```

```
# ข) คะแนนเกินช่วง → 422
curl -X POST ... -d '{\"code\":\"66010055\",\"name\":\"AB Test\",\"email\":\"ab@dpu.ac.th\",\"score\":101}'
#   → {"field":"score","message":"คะแนนต้องไม่เกิน 100"}

# ค) ส่งฟิลด์ที่ระบบไม่รู้จัก → 422  (นี่คือผลของ strictObject)
curl -X POST ... -d '{\"code\":\"66010077\",\"name\":\"AB Test\",\"email\":\"x@dpu.ac.th\",\"nickname\":\"hack\"}'
#   → "ส่งฟิลด์ที่ระบบไม่รู้จักมาด้วย — ตรวจชื่อฟิลด์ให้ตรงกับเอกสาร"

# ง) PATCH โดยไม่ส่งฟิลด์อะไรมาเลย → 422  (นี่คือผลของ .refine)
curl -X PATCH http://localhost:3000/api/v1/students/<ID> -H "Content-Type: application/json" -d '{}'
#   → "ต้องส่งอย่างน้อยหนึ่งฟิลด์ที่ต้องการแก้"
```

**สังเกตว่าทั้ง 4 กรณีไม่มีคำขอไหนแตะฐานข้อมูลเลย** — เปิด log ดูได้ ไม่มี query ออกไป
**นี่คือมูลค่าที่วัดได้ของชั้นที่ 1**

# ขั้นที่ 11.2 — ลองทำให้พัง (ชั้นที่ 3 และการหาไม่เจอ)

```
# จ) รหัสนักศึกษาซ้ำ → 409  (ข้อมูลถูกต้องทุกอย่าง แต่ชนกับของที่มีอยู่)
curl -X POST http://localhost:3000/api/v1/students -H "Content-Type: application/json" `
  -d '{\"code\":\"66010001\",\"name\":\"Duplicate Test\",\"email\":\"dup@dpu.ac.th\"}'
#   → {"error":{"code":"DUPLICATE","message":"ข้อมูลนี้มีอยู่แล้วในระบบ","details":{"field":"code"}}}

# ฉ) uuid ถูกรูปแบบ แต่ไม่มีแถวนี้ → 404
curl http://localhost:3000/api/v1/students/11111111-1111-4111-8111-111111111111
#   → {"error":{"code":"NOT_FOUND","message":"ไม่พบนักศึกษา id 11111111-..."}}

# ช) id ไม่ใช่ uuid → 400  (ไม่ใช่ 422 เพราะผิดที่ path)
curl http://localhost:3000/api/v1/students/abc
#   → {"error":{"code":"VALIDATION_ERROR",...,"details":[{"field":"id","message":"id ต้องเป็น UUID ..."}]}}

# ซ) ลบซ้ำครั้งที่สอง → 404  (Prisma P2025)
curl -X DELETE http://localhost:3000/api/v1/students/<ID ที่ลบไปแล้ว>
#   → {"error":{"code":"NOT_FOUND","message":"ไม่พบข้อมูลที่ต้องการแก้ไขหรือลบ"}}

# ฌ) limit เกินเพดาน → 400
curl "http://localhost:3000/api/v1/students?limit=999"
#   → {"field":"limit","message":"limit สูงสุด 100"}

# ญ) ไม่มีเส้นทางนี้ → 404 ROUTE_NOT_FOUND
curl http://localhost:3000/api/v1/teachers
```

**ทุกคำตอบใช้รูปแบบ `{ error: { code, message, details? } }` เหมือนกันหมด**
ทั้งที่ต้นทางมาจาก Zod · จาก `AppError` ที่เราโยน · และจากฐานข้อมูล
**นั่นคือผลงานของตัวดักรวมศูนย์ไฟล์เดียว**

# คิดก่อนดูเฉลย

จากที่ทดลองมาทั้งขั้นที่ 11

### คำถามที่ 1

กรณี **จ) รหัสนักศึกษาซ้ำ** เราตอบ `409` ไม่ใช่ `422` — ทำไม?
ทั้งสองกรณีก็คือ "สร้างไม่สำเร็จ" เหมือนกัน

### คำถามที่ 2

กรณี **ช)** `GET /students/abc` เราตอบ `400` ไม่ใช่ `404`
แต่ URL นี้ก็ "ไม่มีอยู่" จริง ๆ นี่นา — ทำไมไม่ใช่ `404` ?

### คำถามที่ 3

ถ้าเราลบ `@unique` ออกจาก `code` ใน `schema.prisma` แล้วสั่ง `migrate dev` ใหม่
**กรณี จ) จะได้ผลอะไร** และเป็นปัญหาตรงไหน?

> **ข้อ 3 คือบทสรุปของคาบนี้ทั้งคาบ** — ลองตอบให้ได้ก่อนเลื่อนต่อ

# เฉลย

## ข้อ 1 — เพราะ ข้อมูลที่ส่งมาถูกต้องทุกอย่าง

`422` แปลว่า "เนื้อหาในคำขอใช้ไม่ได้" — แก้ข้อมูลแล้วยิงใหม่ได้

`409` แปลว่า **"ข้อมูลของคุณไม่มีอะไรผิด แต่ชนกับสถานะปัจจุบันของระบบ"**

รหัส `66010001` เป็นรหัสที่ถูกต้องสมบูรณ์ — มันแค่ **มีคนใช้ไปแล้ว**

ความต่างนี้สำคัญกับฝั่งหน้าเว็บ: `422` ขึ้นข้อความใต้ช่องกรอก · `409` ต้องบอกว่า "รหัสนี้ถูกใช้แล้ว"

## ข้อ 2 — เพราะ คำขอเขียนมาผิดรูปตั้งแต่ต้น

`404` แปลว่า "ที่อยู่นี้ถูกต้องตามรูปแบบ แต่ไม่มีของอยู่ที่นั้น" — ถ้าตอบ `404` ให้ `abc`

เราจะแยกไม่ออกระหว่าง "พิมพ์ id ผิดรูป" กับ "id ถูกแต่ข้อมูลถูกลบไปแล้ว"

ซึ่งเป็นสองปัญหาที่ต้องแก้ต่างกันคนละแบบ

## ข้อ 3 — จะได้ `201` และมีแถวรหัส `66010001` สองแถวในตาราง ❌

| ผลที่ตามมา | ความเสียหาย |
|---|---|
| `GET /students?q=66010001` คืนสองแถว | หน้าเว็บไม่รู้ว่าจะแสดงคนไหน |
| ประกาศคะแนนอ้างอิงจากรหัสนักศึกษา | **คะแนนไปผิดคน** |

# โครงสร้างไฟล์สุดท้าย — เทียบกับของคุณ

```
ce385-week8/
├─ .env                        ← ห้าม commit
├─ .env.example
├─ .gitignore                  .env · node_modules · dist · src/generated
├─ package.json
├─ package-lock.json
├─ tsconfig.json
├─ prisma.config.ts
├─ prisma/
│  ├─ schema.prisma
│  ├─ seed.ts
│  └─ migrations/
│     ├─ migration_lock.toml
│     └─ 20260927070235_init/migration.sql     ← commit ขึ้น Git ด้วย
└─ src/
   ├─ server.ts
   ├─ app.ts
   ├─ lib/prisma.ts
   ├─ schemas/student.schema.ts
   ├─ services/student.service.ts
   ├─ routes/students.route.ts
   ├─ errors/app-error.ts
   ├─ middlewares/
   │  ├─ logger.ts
   │  ├─ validate.ts
   │  ├─ async-handler.ts
   │  └─ error-handler.ts
   └─ generated/prisma/                        ← Prisma สร้างให้ · ห้าม commit · ห้ามแก้
```

**13 ไฟล์ที่เราเขียนเอง** · ทุกไฟล์มีหน้าที่เดียว · ไฟล์ที่ยาวที่สุดไม่ถึง 100 บรรทัด

# ผ่านแล้วหรือยัง — ตรวจตัวเอง 12 ข้อ

| ✓ | ข้อ |
|---|---|
| ☐ | `node --version` ได้ v20 ขึ้นไป และ `Get-Service postgresql*` เป็น Running |
| ☐ | ฐานข้อมูล `ce385_week8` มีอยู่จริง (เห็นใน pgAdmin หรือ Prisma Studio) |
| ☐ | `npx prisma migrate dev` ผ่าน และมีไฟล์ `migration.sql` เกิดขึ้น |
| ☐ | เปิด `migration.sql` แล้วเห็น `CREATE UNIQUE INDEX` สองบรรทัด |
| ☐ | `npx prisma db seed` ได้ข้อความ "เพิ่มข้อมูลนักศึกษาแล้ว 5 คน" |
| ☐ | `npm run typecheck` ไม่มี error แม้แต่ข้อเดียว |
| ☐ | `npm run dev` ขึ้นข้อความ "เซิร์ฟเวอร์พร้อมรับคำขอ" |
| ☐ | `GET /students?page=1&limit=3` ได้ 200 และ `meta.total` = 5 |
| ☐ | `POST` ได้ **201** และมี header **`Location`** |
| ☐ | `DELETE` ได้ **204** และไม่มีเนื้อหาตอบกลับ |
| ☐ | `POST {}` ได้ **422** พร้อม `details` ที่บอกครบ 3 ฟิลด์ |
| ☐ | `POST` รหัสซ้ำได้ **409** และ `GET /students/abc` ได้ **400** |

**ติ๊กครบ 12 ข้อ = จบเป้าหมายของคาบนี้**
ข้อที่ติ๊กไม่ได้ ให้ไปดูตารางแก้ปัญหาในสไลด์ถัดไปก่อนถาม

# ตารางแก้ปัญหา (1/2) — ตอนติดตั้งและตั้งค่า

| อาการ | สาเหตุ | วิธีแก้ |
|---|---|---|
| `'node' is not recognized` | ยังไม่ปิด-เปิดหน้าต่างใหม่หลังติดตั้ง | ปิด PowerShell ทุกหน้าต่างแล้วเปิดใหม่ |
| `P1001: Can't reach database server at localhost:5432` | PostgreSQL ไม่ได้รัน | `Get-Service postgresql*` → ถ้าไม่ Running สั่ง `Start-Service postgresql-x64-17` |
| `P1000: Authentication failed` | รหัสผ่านใน `.env` ผิด | ตรวจรหัสที่จดไว้ตอนติดตั้ง · ถ้าลืมต้องตั้งใหม่ผ่าน pgAdmin |
| `P1003: Database does not exist` | ยังไม่สร้างฐานข้อมูล | สร้างใน pgAdmin หรือปล่อยให้ `migrate dev` สร้างให้ |
| `ไม่พบค่า DATABASE_URL` | ยังไม่มีไฟล์ `.env` หรือวางผิดที่ | ต้องอยู่ระดับเดียวกับ `package.json` |
| `Environment variable not found: DATABASE_URL` (จาก prisma) | `prisma.config.ts` ไม่มี `import "dotenv/config"` | เพิ่มบรรทัดนั้นเป็นบรรทัดแรกของไฟล์ |
| `psql` ใช้ไม่ได้ | ไม่ได้ติ๊ก Command Line Tools ตอนติดตั้ง | ใช้ path เต็ม `& "C:\Program Files\PostgreSQL\17\bin\psql.exe"` |
| ติดตั้ง PostgreSQL ไม่ผ่านเลย | สิทธิ์ผู้ใช้ · โปรแกรมป้องกันไวรัส | **ใช้ Neon ไปก่อน** (ขั้นที่ 0.4) แล้วค่อยกลับมาแก้ |
| `too many clients already` | เปิดโปรแกรมทิ้งไว้หลายตัวไม่ได้ปิด | ปิดหน้าต่างที่รันค้าง · รีสตาร์ท service ของ PostgreSQL |

# ตารางแก้ปัญหา (2/2) — ตอนเขียนโค้ดและรัน

| อาการ | สาเหตุ | วิธีแก้ |
|---|---|---|
| `Cannot find module './app'` | ลืมใส่ `.js` ท้าย import | **ทุก import ในไฟล์เราต้องลงท้าย `.js`** (ไม่ใช่ `.ts`) |
| `Cannot find module '../generated/prisma/client.js'` | ยังไม่ generate หรือ generate ไปคนละที่ | `npx prisma generate` · ตรวจ `output` ใน `schema.prisma` |
| `prisma.student` ไม่มีใน autocomplete | Client เก่า ยังไม่รู้จักโมเดลใหม่ | `npx prisma generate` แล้ว **รีสตาร์ท TS Server ใน VS Code** |
| `EADDRINUSE :::3000` | พอร์ตถูกใช้อยู่ | ปิดโปรแกรมเดิม · หรือแก้ `PORT` ใน `.env` |
| `req.body` เป็น `undefined` | ลืม `app.use(express.json())` | เพิ่มเข้าไปเป็น middleware ตัวแรกใน `app.ts` |
| ยิงแล้วค้างไม่มีคำตอบ | ลืม `asyncHandler` ห่อ handler แบบ `async` | ห่อทุก handler ที่มี `await` ด้วย `asyncHandler` |
| ทุก error กลายเป็น 500 | ลำดับ `if` ใน `error-handler` ผิด หรือลืม `instanceof` | ตรวจว่า `ZodError` และ `AppError` ถูกเช็กก่อนข้อ 4 |
| error ไม่เข้า `errorHandler` เลย | วาง `errorHandler` ไม่ใช่ตัวสุดท้ายใน `app.ts` | ย้ายไปไว้บรรทัดสุดท้าย |
| `?page=2` ฟ้องว่าไม่ใช่ตัวเลข | ลืม `z.coerce` | ใช้ `z.coerce.number()` กับทุกค่าที่มาจาก query |
| JSON ใน `curl` พังบน PowerShell | เครื่องหมายคำพูดไม่ได้ escape | ใช้ `\"` · หรือเปลี่ยนไปใช้ **Postman** |
| แก้ `schema.prisma` แล้วตารางไม่เปลี่ยน | ลืมสั่ง migrate | `npx prisma migrate dev --name <ชื่อ>` ทุกครั้งที่แก้ schema |

# สรุปสิ่งที่ได้จากคาบนี้

| เรื่อง | ใจความที่ต้องจำ |
|---|---|
| **REST** | คำนามอยู่ที่ URL · คำกริยาอยู่ที่ method · ความสม่ำเสมอมีค่ากว่าความสวย |
| **Status Code** | `201` ต้องมี `Location` · `204` ห้ามมี body · `4xx` คุณผิด `5xx` เราผิด |
| `400` **vs** `422` | ผิดที่ path/query → 400 · ผิดที่ body → 422 · **เลือกแล้วใช้เหมือนกันทั้งระบบ** |
| `409` | ข้อมูลถูกต้องทุกอย่าง แต่ชนกับสถานะปัจจุบันของระบบ |
| **Validation** | Fail-fast ที่ขอบระบบ · **อย่าเชื่อข้อมูลจากภายนอกแม้แต่ตัวอักษรเดียว** |
| **ป้องกัน 3 ชั้น** | Zod (รูปร่าง) → service (กฎธุรกิจ) → **ฐานข้อมูล (สิ่งที่โกงไม่ได้)** |
| **Zod** | `z.infer` ทำให้กติกาตอนรันกับชนิดข้อมูลตอนเขียนเป็นสิ่งเดียวกัน |
| **ORM** | ใช้เป็นค่าตั้งต้น · ลงไปเขียน SQL เองเฉพาะที่จำเป็น · **ต้องอ่าน SQL ที่มันสร้างได้** |
| **Prisma** | `schema` → `migrate` (ฐานข้อมูล) + `generate` (TypeScript) — **ต้องทำทั้งสองทาง** |
| **Migration** | คือ Git ของฐานข้อมูล · commit ทุกครั้ง · ห้ามแก้ไฟล์ที่ apply ไปแล้ว |
| **แบ่งชั้น** | `route` (HTTP) → `service` (ตรรกะ) → `prisma` (ข้อมูล) · **service ห้ามรู้จัก express** |
| **จัดการ error** | มี **ที่เดียว** ในระบบที่แปลง error เป็น HTTP · ไม่รั่วรายละเอียดภายในออกไป |

**ประโยคเดียวที่สรุปทั้งคาบ**

# สัปดาห์หน้า

## สัปดาห์ที่ 9 — สอบกลางภาค

ครอบคลุมเนื้อหาสัปดาห์ที่ 2–8 · **20 คะแนน** · เวลา 2 ชั่วโมง
รูปแบบ: ปรนัย + อัตนัยเชิงออกแบบ

**สิ่งที่ต้องเตรียม**

ทบทวนการเลือก HTTP method และ status code ให้ถูกความหมาย
ฝึกออกแบบชุด endpoint จากโจทย์ · อธิบายเหตุผลของการแบ่งชั้นได้

**เก็บโฟลเดอร์** `ce385-week8` **ไว้ให้ดี** — จะได้ใช้ต่อหลังสอบ